

Supervised Learning -Classification

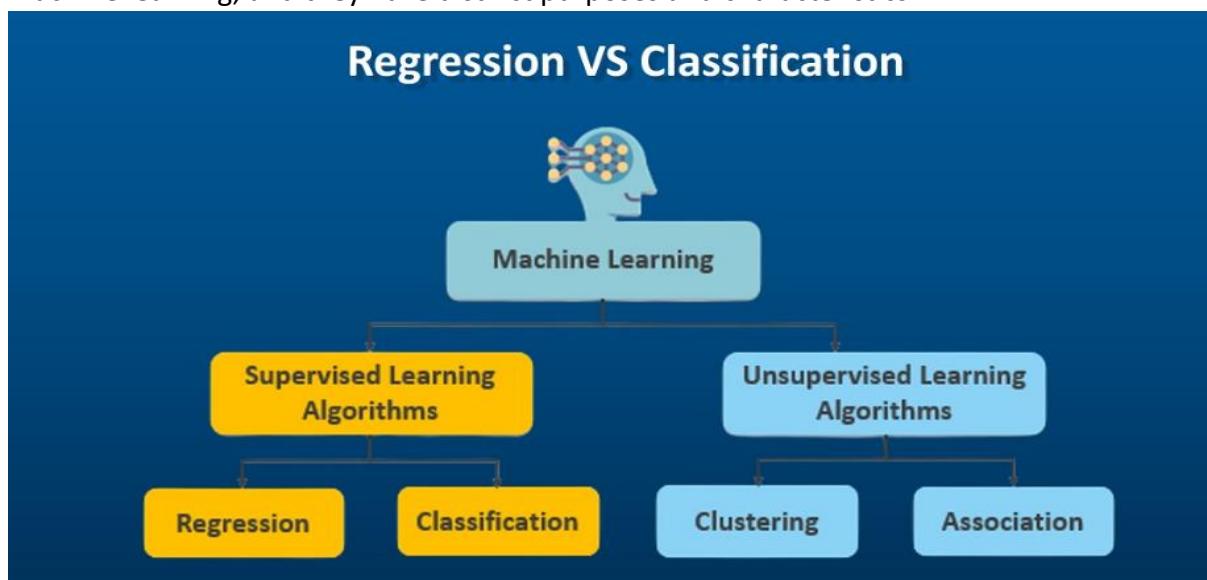
Unit-4

Session-1

Classification problem- Probability based approach

- Definition
- Types of classification problems
- Examples of classification tasks
- Difference between classification and regression

Regression and classification are two fundamental types of supervised learning tasks in machine learning, and they have distinct purposes and characteristics.



1. Introduction to Classification

Classification is a supervised learning task where the goal is to predict the class or category of given data points. It involves assigning labels to inputs based on their features.

Key Features of Classification:

- **Input:** Features (numerical, categorical, or mixed data).
- **Output:** Discrete class labels (e.g., spam or not spam, 0 or 1).
- **Types:**
 - **Binary Classification:** Two classes (e.g., fraud detection).
 - **Multi-class Classification:** More than two classes (e.g., digit recognition).

Example: Predicting whether a patient has a disease (Yes/No).

CLASSIFICATION VS REGRESSION



Student Profile



Predicting Student
Pass Or Fail



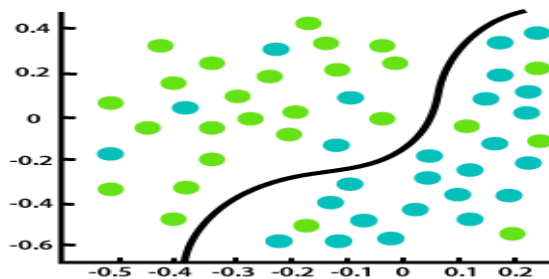
Student Profile



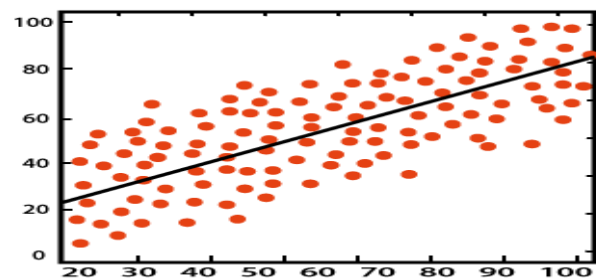
Predicting Student Marks
Percentage

Regression	Classification
The task of predicting a continuous quantity is known as regression.	The classification process involves anticipating a discrete class label.
The task is to map the input value (x) to the continuous output variable (y).	The task is to map the input value(x) to the discrete output variable (y).
A regression model can predict a discrete value, but only in the form of an integer quantity.	A classification model can predict a continuous value, but it is in the form of a probability for a class label.
The output variable in regression must be of continuous value.	The output variable in Classification must be discrete.

We can call a Logistic Regression a Linear Regression model but the Logistic Regression uses a more complex cost function, this cost function can be defined as the '**Sigmoid function**' or also known as the 'logistic function' instead of a linear function.



Classification

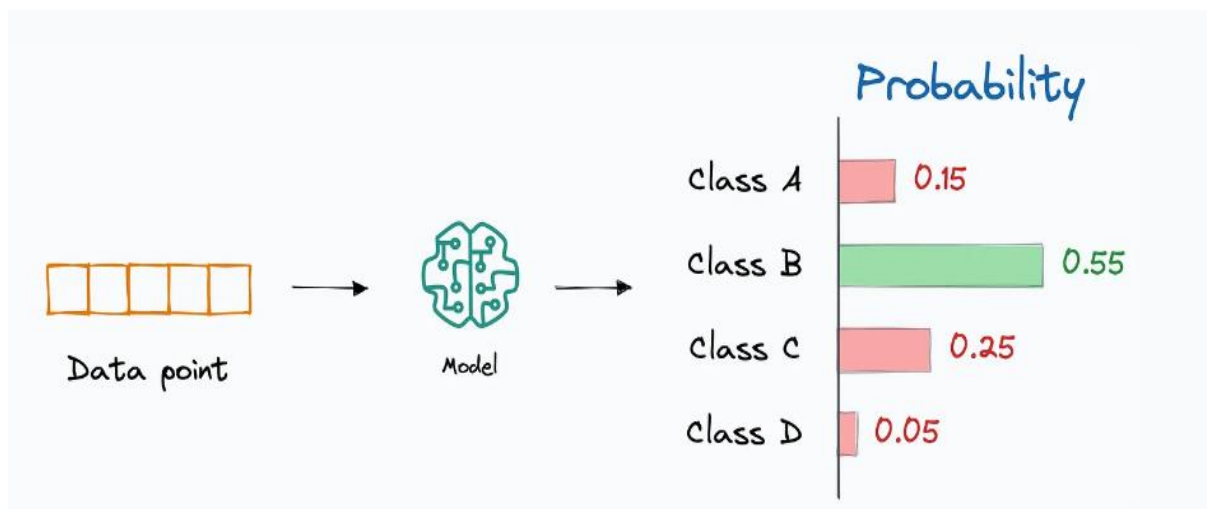


Regression

2. Probability-Based Classification

In probability-based classification, the model estimates the likelihood that an input belongs to a particular class and assigns it to the most probable class.

- **Output:** A probability score for each class (e.g., 70% spam, 30% not spam).
- **Advantages:**
 - Provides confidence levels for predictions.
 - Useful for threshold-based decision-making.



3. Bayesian Probability Basics

Probability plays a foundational role in classification. Bayesian methods use probabilities to make predictions.

Key Concepts:

- **Joint Probability** $P(A,B)$: Probability of both events A and B.
- **Marginal Probability** $P(A)$: Probability of an event irrespective of others.
- **Conditional Probability** $P(A|B)$: Probability of A given B.

Bayes' Theorem:

$$P(A|B) = \frac{P(A) \cdot P(B|A)}{P(B)}$$

$P(A)$ = The probability of A occurring

$P(B)$ = The probability of B occurring

$P(A|B)$ = The probability of A given B

$P(B|A)$ = The probability of B given A

- **Example:** In medical diagnosis, $P(\text{Disease}|\text{Symptoms})$ is derived using $P(\text{Symptoms}|\text{Disease})$.

4. Logistic Regression as a Probability-Based Approach

Logistic regression predicts the probability of a binary outcome using the logistic function.

Key Concepts:

- **Sigmoid Function:**
- The sigmoid maps any real number into the range $[0, 1]$.
- **Decision Rule:** Assign the class based on a threshold (commonly 0.5).

5. Naive Bayes Classifier

Naive Bayes is a family of probabilistic classifiers based on Bayes' Theorem, with an assumption of independence between features. There are different variations of Naive Bayes classifiers, tailored for specific types of data:

Naive Bayes is based on Bayes' theorem with the assumption that features are independent given the class.

Formula:

$$P(C|X) = \frac{P(X|C)P(C)}{P(X)}$$

Where:

- C: Class
- X: Feature set

Types:

1. **Gaussian Naive Bayes:** Assumes normal distribution of features. Continuous Values
2. **Multinomial Naive Bayes:** Used for discrete data like word counts.
3. **Bernoulli Naive Bayes:** Used for binary features.

Aspect	Gaussian	Multinomial	Bernoulli
Data Type	Continuous (numerical).	Discrete (counts).	Binary (presence/absence).
Use Case	Features like age, height.	Text classification (word counts).	Text classification (binary features).
Likelihood Function	Gaussian distribution.	Multinomial distribution.	Bernoulli distribution.
Typical Application	Medical diagnosis, sensor data.	Text analytics (e.g., news articles).	Binary text classification (e.g., spam detection).

6. Evaluation Metrics for Classification

Metrics:

- **Accuracy:** Ratio of correct predictions to total predictions.
- **Precision:** Proportion of true positives among predicted positives.
- **Recall (Sensitivity):** Proportion of true positives among actual positives.
- **F1-Score:** Harmonic mean of precision and recall.

ROC Curve: Receiver Operating Characteristic Curve

A plot of the True Positive Rate (TPR) vs. False Positive Rate (FPR), showing the tradeoff between sensitivity and specificity.

AUC measures the overall performance.

Supervised Learning -Classification

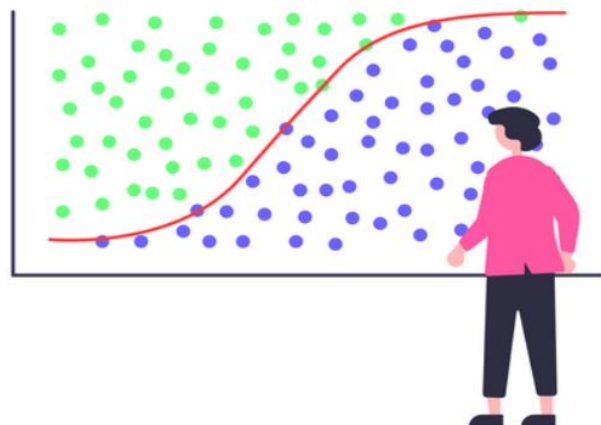
Unit-4

Session-2

Logistic Regression- log-odd, sigmoid transformation

- Introduction to Logistic Regression
- The Problem with Linear Regression for Classification
- Log-Odds in Logistic Regression
- Sigmoid Function and Transformation
- Relationship Between Log-Odds and Probability
- Logistic Regression Model
- Decision Boundary in Logistic Regression
- Applications of Logistic Regression

WHAT IS
LOGISTIC
REGRESSION?



1. Introduction to Logistic Regression

Logistic regression is a supervised learning algorithm used for binary classification. It predicts the probability of a data point belonging to one of two classes.

- Input: Features $(x_1, x_2, \dots, x_n)$.
- Output: A probability score (P) that an input belongs to a class.

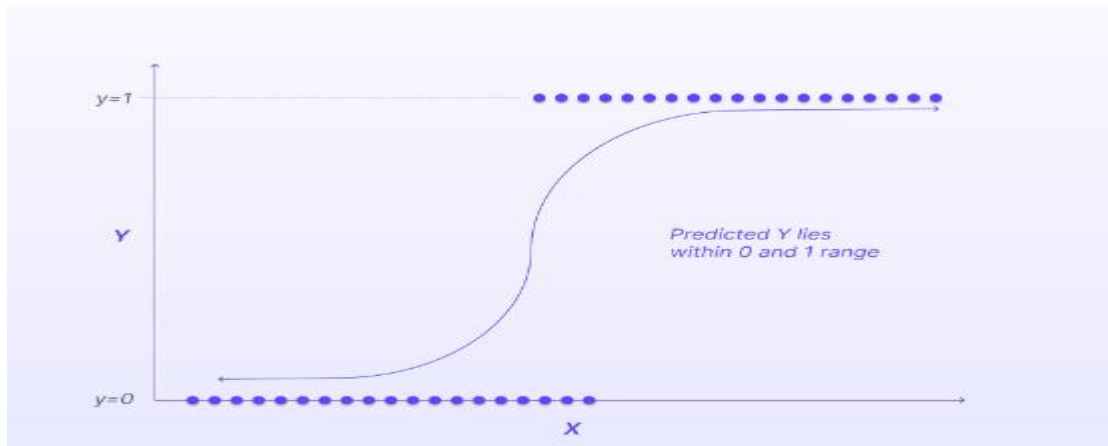
Example: Predict whether an email is spam (1) or not spam (0).

2. The Problem with Linear Regression for Classification

Linear regression is unsuitable for classification because:

- It predicts continuous values, which are unbounded.
- Outputs cannot be interpreted as probabilities.

Solution: Logistic regression uses a sigmoid function to constrain predictions to the range [0, 1], making them interpretable as probabilities.



3. Log-Odds in Logistic Regression

Logistic regression models the log-odds of an event.

Odds

The odds represent the ratio of the probability of success (P) to the probability of failure (1-P):

$$\text{Odds} = \frac{P}{1 - P}$$

Log-Odds

The log-odds, also known as the logit, are the natural logarithm of the odds:

$$\text{Log-Odds (Logit)} = \ln \left(\frac{P}{1 - P} \right)$$

In logistic regression, the log-odds are modeled as a linear combination of the input features:

$$\ln \left(\frac{P}{1 - P} \right) = \beta_0 + \beta_1 x_1 + \dots + \beta_n x_n$$

Here:

- β_0 : Intercept term.
- $\beta_1, \beta_2, \dots, \beta_n$: Coefficients for the features.

4. Sigmoid Function and Transformation

The sigmoid function maps the log-odds to a probability between 0 and 1.

Sigmoid Function Formula:

$$h(x) = \frac{1}{1 + e^{-z}}, \quad \text{where } z = \beta_0 + \beta_1 x_1 + \dots + \beta_n x_n$$

- z : Log-odds (linear combination of features).
- $h(x)$: Predicted probability of the positive class.

Properties of the Sigmoid Function

$$\begin{aligned} e^{-\infty} &= 0 \\ e^{\infty} &= \infty \end{aligned}$$

- $h(x) \rightarrow 1$ as $z \rightarrow +\infty$
- $h(x) \rightarrow 0$ as $z \rightarrow -\infty$
- $h(x) = 0.5$ when $z = 0$.

- **5. Relationship Between Log-Odds and Probability**

- The probability (P) can be expressed in terms of the log-odds:

$$P = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1 + \dots + \beta_n x_n)}}$$

- Similarly, the log-odds can be obtained from the probability:

$$\ln \left(\frac{P}{1 - P} \right) = \beta_0 + \beta_1 x_1 + \dots + \beta_n x_n$$

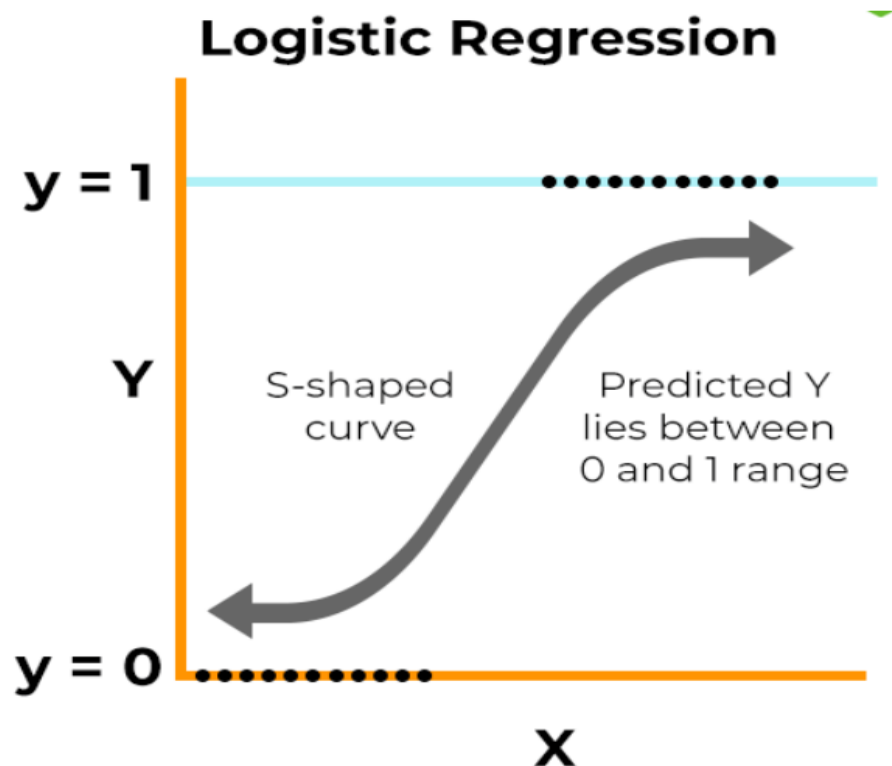
6. Logistic Regression Model

Logistic regression predicts the class based on a probability threshold:

- If $P \geq 0.5$, predict class 1.
- If $P < 0.5$, predict class 0.

Steps in Logistic Regression:

1. Compute the log-odds ($z = \beta_0 + \beta_1 x_1 + \dots + \beta_n x_n$).
2. Apply the sigmoid function to obtain the probability (P).
3. Classify based on the threshold.



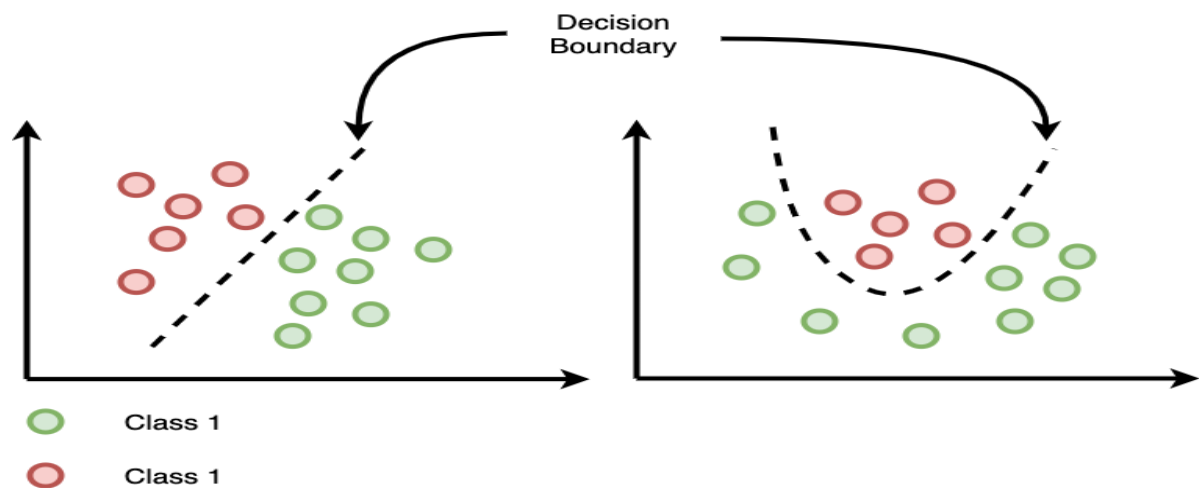
7. Decision Boundary in Logistic Regression

The decision boundary is the point where the predicted probability is 0.5:

$$\ln \left(\frac{P}{1-P} \right) = 0 \quad \Rightarrow \quad \beta_0 + \beta_1 x_1 + \dots + \beta_n x_n = 0$$

For binary features, this is a straight line in 2D space.

For more complex data, the boundary can be nonlinear.



8. Applications of Logistic Regression

- Medical Diagnosis: Predicting whether a patient has a disease.
- Credit Scoring: Determining the likelihood of loan default.
- Spam Detection: Classifying emails as spam or not spam.

Example: Medical Diagnosis: Predicting Disease Presence

- Purpose: Logistic regression can help predict whether a patient has a particular disease based on medical data.
- Process:
 - Medical features like age, blood pressure, cholesterol levels, and test results are used as input variables.
 - Logistic regression predicts the probability of disease presence (e.g., heart disease, diabetes).
- Outcome:
 - The output is a probability score between 0 and 1.
 - Threshold-based classification determines the outcome (e.g., probability > 0.5 indicates disease presence).

Supervised Learning -Classification

Unit-4

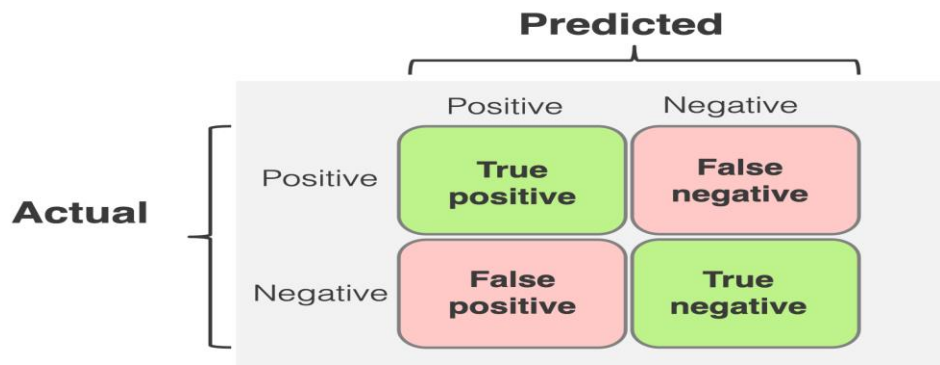
Session-3

Classification Metrics

1. Confusion Matrix
2. Accuracy
3. Error Rate
4. Precision
5. Recall
6. ROC curve
7. F1 score

Confusion Matrix

A **confusion matrix** is a table that summarizes the performance of a classification model by comparing predicted and actual values. It is especially useful for binary and multiclass classification problems.

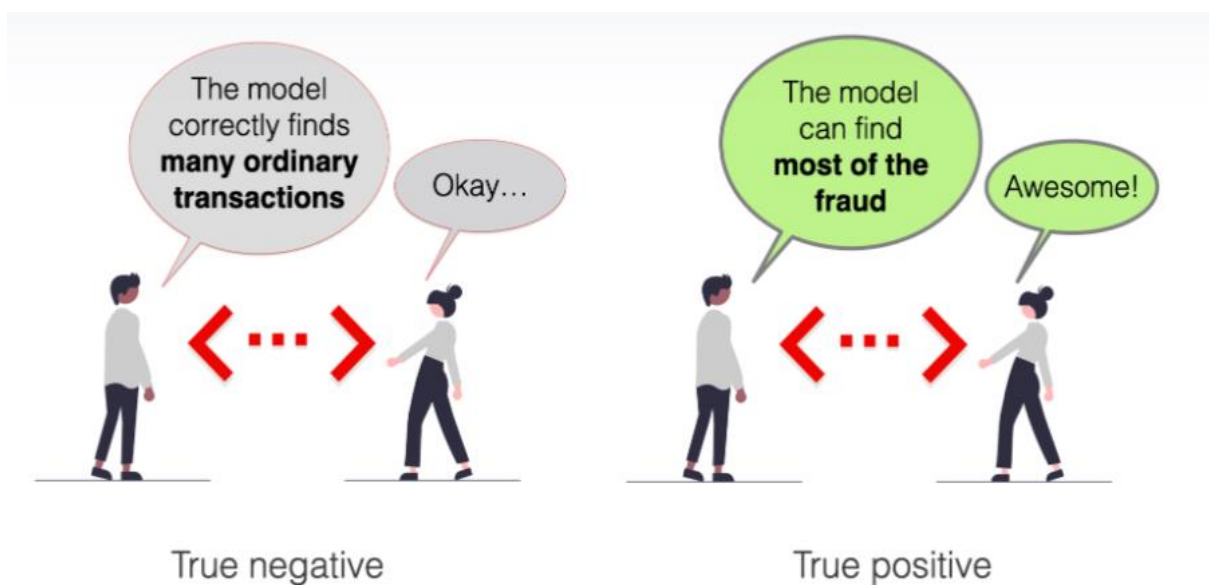


Structure of a Confusion Matrix (Binary Classification)

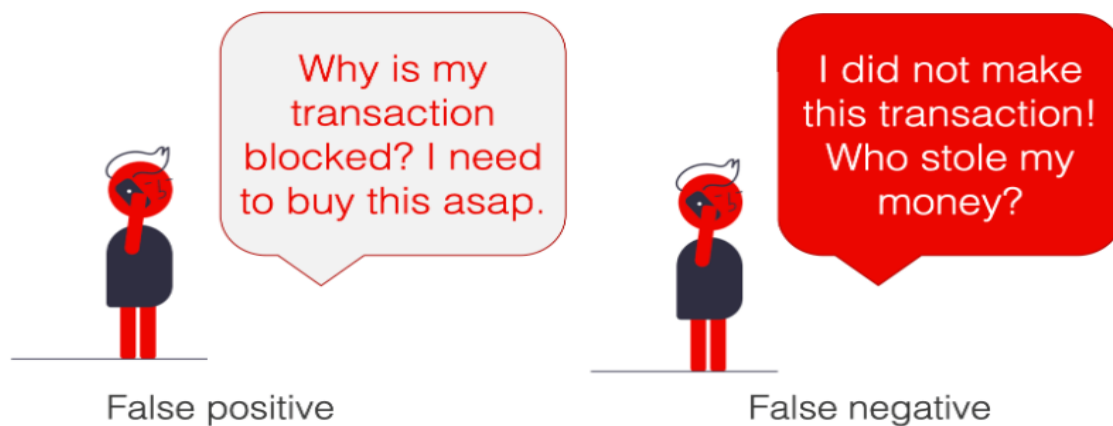
	Predicted Positive	Predicted Negative
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

- **True Positive (TP):** Cases where the model correctly predicts the positive class.
- **False Positive (FP):** Cases where the model incorrectly predicts the positive class (Type I error).
- **False Negative (FN):** Cases where the model incorrectly predicts the negative class (Type II error).
- **True Negative (TN):** Cases where the model correctly predicts the negative class.

	Predicted	Actual	
1.	Not fraud	Not fraud	True Negative
2.	Not fraud	Not fraud	
3.	Not fraud	Fraud	True Positive
4.	Fraud	Fraud	
n.	Fraud	Not fraud	



	Predicted	Actual	
1.	Not fraud	Not fraud	
2.	Not fraud	Not fraud	
3.	Not fraud	Fraud	False Negative
4.	Fraud	Fraud	
n.	Fraud	Not fraud	False Positive



1. Accuracy

Definition: Measures the proportion of correctly classified instances out of the total instances.

Formula:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

When to Use: Suitable when classes are balanced.

Limitation: Misleading for imbalanced datasets (e.g., detecting rare diseases).

2. Precision (Positive Predictive Value)

Definition: Measures the percentage of positive predictions that are correct.

Formula:

$$\text{Precision} = \frac{TP}{TP + FP}$$

When to Use: Important when false positives need to be minimized (e.g., spam detection).

3. Recall (Sensitivity, True Positive Rate)

Definition: Measures the percentage of actual positive instances that are correctly predicted.

Formula:

$$\text{Recall} = \frac{TP}{TP + FN}$$

When to Use: Crucial when false negatives need to be minimized (e.g., cancer detection).

4. F1 Score

Definition: Harmonic mean of precision and recall, balancing their trade-off.

Formula:

$$\text{F1 Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

When to Use: Useful when you need a balance between precision and recall, especially in imbalanced datasets.

5. Specificity (True Negative Rate)

Definition: Measures the percentage of actual negatives correctly predicted.

Formula:

$$\text{Specificity} = \frac{\text{TN}}{\text{TN} + \text{FP}}$$

When to Use: Relevant when correctly identifying negatives is as important as identifying positives.

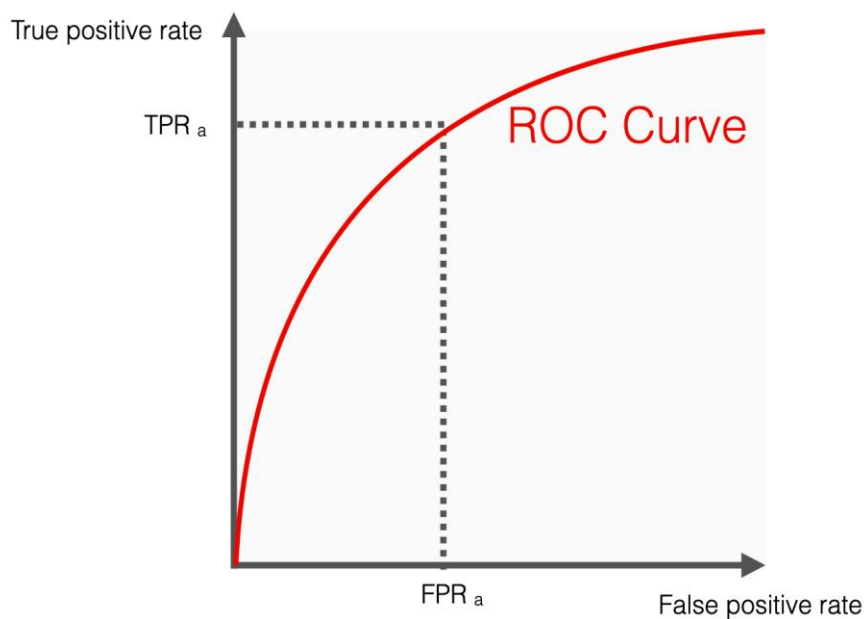
6. ROC-AUC (Receiver Operating Characteristic - Area Under Curve)

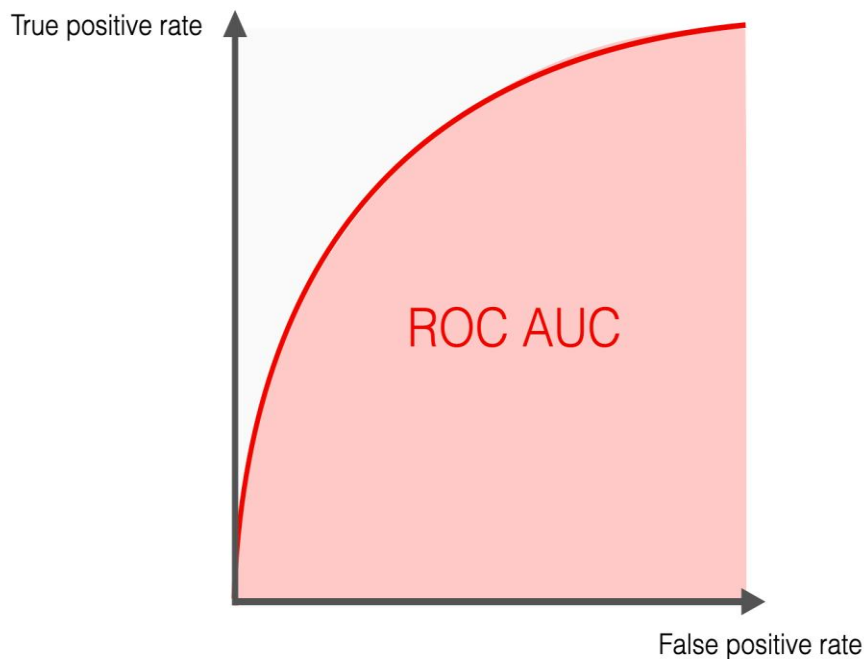
- **Definition:** Evaluates the trade-off between true positive rate (recall) and false positive rate across various thresholds.
- **Components:**
 - **ROC Curve:** Plots TPR (y-axis) vs. FPR (x-axis). $\text{TP} / (\text{TP} + \text{FN})$ vs. $\text{FP} / (\text{FP} + \text{TN})$
 - **AUC:** Area under the ROC curve; a higher value indicates better model performance.
- **When to Use:** Suitable for binary classification, regardless of class balance.

Classification threshold

You might ask, what do "different" TPR and FPR values mean? Did we not just calculate them once and for all?

- In fact, we calculated the values for a given confusion matrix **at a given decision threshold**. But for a probabilistic classification model, these TPR and FPR values are not set in stone.
- You can vary the decision threshold that defines how to convert the model predictions into labels. This, in turn, can change the number of errors the model makes.

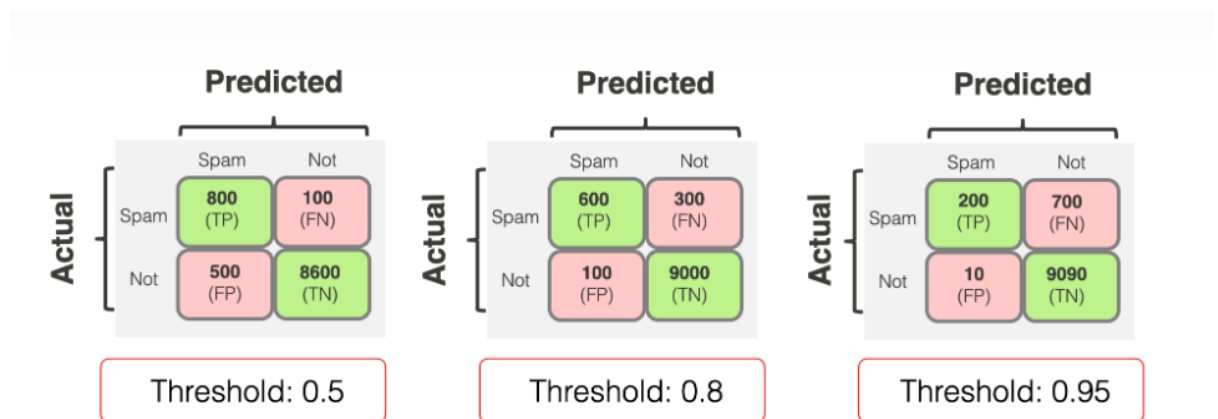




A probabilistic classification model returns a number from 0 to 1 for each object. For example, for each email, it predicts how likely this email is spam. For a given email, it can be 0.1, 0.55, 0.99, or any other number.

You then have to decide at which probability you **convert this prediction to a label**. For instance, you can label all emails with a predicted probability of over 0.5 as spam. Or, you can only apply this decision when the score is 0.8 or higher.

This choice is what sets the classification threshold.



When you **set the threshold higher**, you make the model "more conservative." It assigns the *True* label when it is "more confident." But as a consequence, you typically lower recall: you detect fewer examples of the target class overall.

When you **set the threshold lower**, you make the model "less strict." It assigns the *True* label more often, even when "less confident." Consequently, you increase recall: you will detect more examples

of the target class. However, this may also lead to lower precision, as the model may make more False Positive predictions.

TPR and FPR change in the same direction. The higher the recall (TPR), the higher the rate of false positive errors (FPR). The lower the recall, the fewer false alarms the model gives.

TPR	True Positive Rate	Sensitivity or Recall	$\frac{\text{True Positive}}{\text{Total number of Actual Positives}}$	$\frac{TP}{TP + FN}$
TNR	True Negative Rate	Specificity	$\frac{\text{True Negative}}{\text{Total number of Actual Negatives}}$	$\frac{TN}{TN + FP}$
FPR	False Positive Rate	1-Specificity	$\frac{\text{False Positive}}{\text{Total number of Actual Negatives}}$	$\frac{FP}{TN + FP}$
PPR	Positive Predictive Rate	Precision	$\frac{\text{True Positive}}{\text{Total number of Predicted Positives}}$	$\frac{TP}{TP + FP}$

Accuracy	$\frac{TP + TN}{\text{Total number of points}}$
F1 Score	$\frac{2 * \text{Precision} * \text{Recall}}{\text{Precision} + \text{Recall}}$

Choosing Metrics

- **Balanced Data:** Accuracy, F1 Score, Precision, Recall.
- **Imbalanced Data:** F1 Score, Precision, Recall, MCC, Specificity, ROC-AUC.
- **Probability-Based Models:** Log Loss, ROC-AUC.

Supervised Learning -Classification

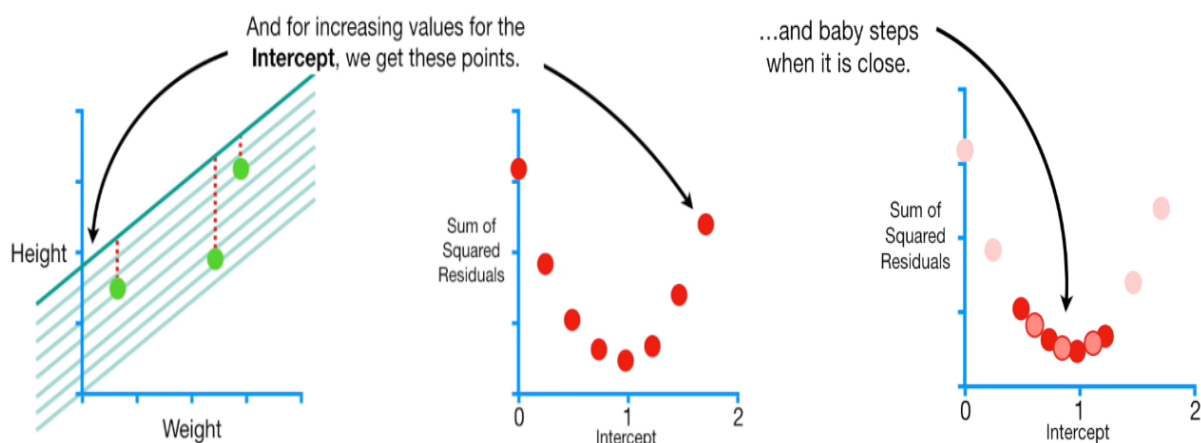
Unit-4

Session-4

Introduction to Gradient Descent

Gradient Descent is an optimization algorithm widely used in machine learning and deep learning to minimize a cost function by iteratively moving in the direction of steepest descent as defined by the negative of the gradient. It is a core concept for training machine learning models, particularly in supervised learning tasks like regression and classification.

In statistics, Machine Learning and other Data Science fields, we optimize a lot of stuff. For example in linear regression, we optimize the Intercept and Slope, or when we use Logistic Regression we optimize the squiggle. A good example is the Sum of the Squared Residuals in Regression: in Machine Learning lingo this is a type of Loss Function. The Residual is the difference between the Observed Value and the Predicted Value.



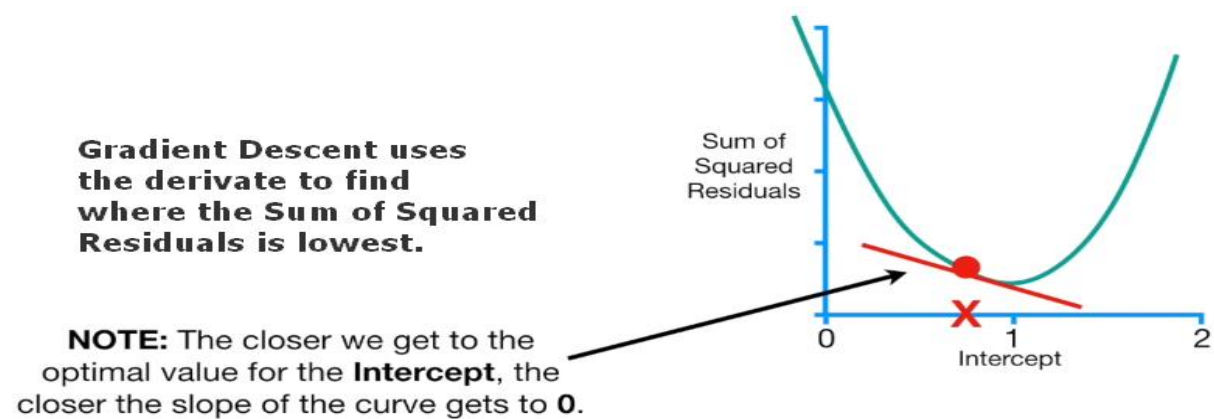
The figure above shows on the y-axis the sum of the squared residuals and the x-axis different value for the intercept. The first point on the y-axis represent the sum of the squared residuals when the intercept is equal to zero. We continue to plot point on the graph based on different value of the intercept. The lowest point in the graph has the lowest sum of the squared residuals. Gradient descent identifies the optimal value by taking big steps when we are far away to the optimal sum of the squared residual, and start to make many steps when it is close to the best solution. Then we can calculate the **derivate d** of each point of the function created by the points. In other words, we are taking the derivative of the **Loss Function**.

1. Why Gradient Descent?

When training machine learning models, the goal is to minimize a loss function, which quantifies the difference between the predicted and actual outputs. Examples of loss functions include:

- **Mean Squared Error (MSE):** Used in regression.
- **Cross-Entropy Loss:** Used in classification.

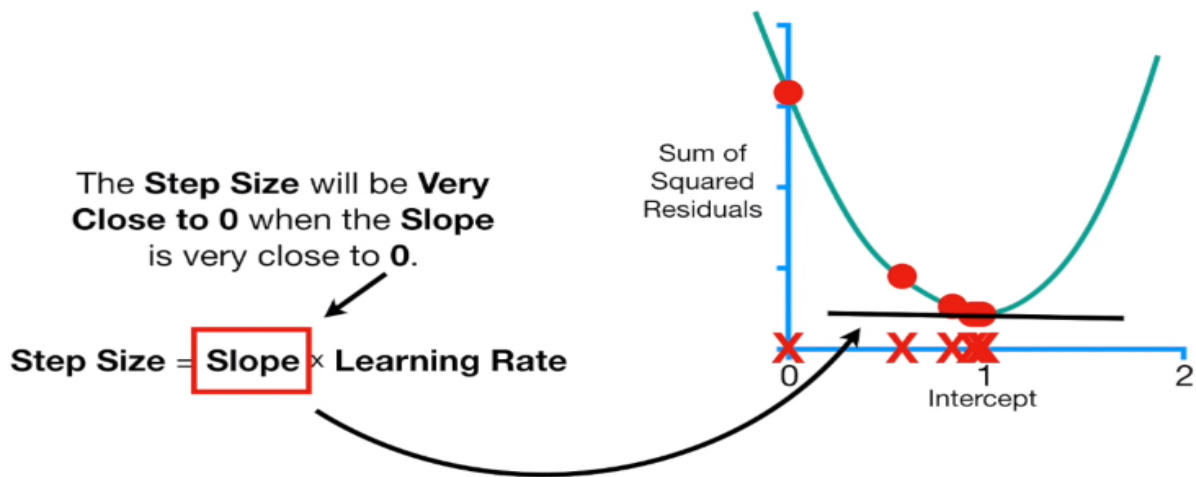
Gradient Descent helps find the set of parameters (weights and biases) that minimize this loss function.



Gradient Descent uses derivative in order to find where the Sum of the Squared Residuals is lowest. The closer we get to the optimal value for the intercept, the closer the slope of the curve gets to zero.

HOW DOES GRADIENT DESCENT KNOW TO STOP TAKING STEPS?

Gradient Descent stops when the step size is very close to zero, and the step size is very close to zero when the **slop size** is close to zero.



- In practice, the **Minimum Step Size** is equal to **0.001** or smaller. Moreover, Gradient Descent includes a limit on the number of steps it will take before giving up. In practice, the **Maximum Number of Steps** is equal to **1000** or greater.
- We can also estimate the intercept and the slope simultaneously. We use the Sum of the Squared Residuals as the Loss Function, and we can represent a **3D graph of the Loss Function** for different values of intercept and the slope.
- We want to find the values for the intercept and slope that give us the minimum Sum of the Squared Residuals.

2. Basic Concept

The process of Gradient Descent involves:

1. Calculating the gradient (partial derivative) of the loss function with respect to the model's parameters.
2. Updating the parameters in the opposite direction of the gradient to reduce the loss.

Mathematically, the update rule for a parameter θ

$$\theta \leftarrow \theta - \alpha \frac{\partial J(\theta)}{\partial \theta}$$

Where:

- α : Learning rate (step size for updates).
- $J(\theta)$: Loss function.

- $\partial J(\theta)/\partial \theta$: Gradient of the loss function with respect to θ .

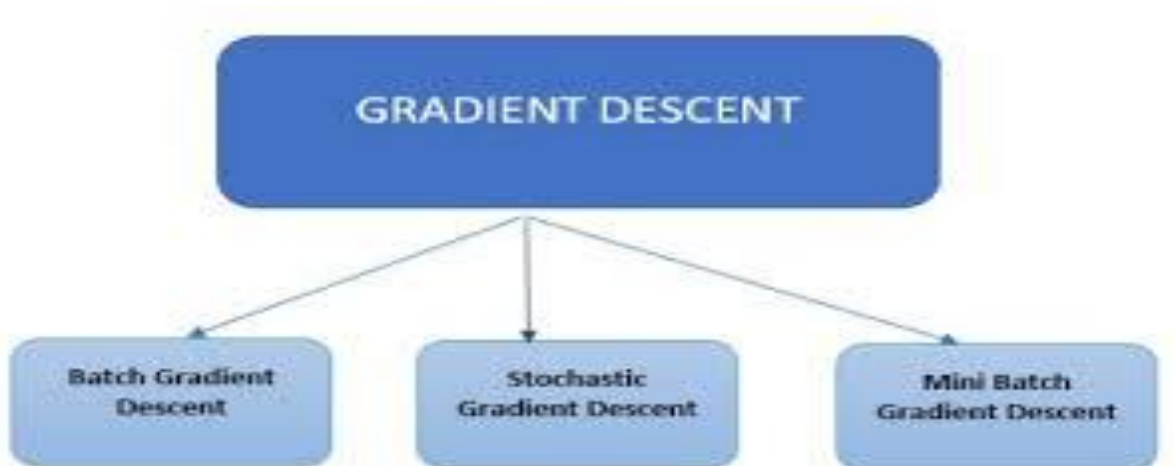
3. Steps in Gradient Descent

1. - Take the derivative of the Loss Function for each parameter in it. i.e take the Gradient of the Loss Function.
- 2 - Pick random values for the parameters.
- 3 - Plug the parameter values into the derivatives (Gradient).
- 4 - Calculate the Step Sizes: Step Size = Slope * Learning Rate.
- 5 - Calculate the New Parameters (e.g. intercept): New Parameter = Old Parameter - Step Size.
- 6 - Go back to step 3 and repeat until Step Size is very small, or when the Maximum Number of Steps is reached.

Learning rate, used in the calculation of the Step Size is a hyper-parameter that controls how much we are adjusting the weights with respect of the Loss Function. The lower the values, the slower we travel along the downward slope. This make sure that we do not miss any **local minima**. Moreover, the learning rate affects how quickly our model can coverge to a local minina (aka arrive at the best accuracy).

4. Types of Gradient Descent

There are three main types of Gradient Descent based on how the data is used:



a) Batch Gradient Descent

- Uses the entire dataset to compute the gradient.

- Pros: Converges steadily and guarantees convergence to a minimum.
- Cons: Computationally expensive for large datasets.

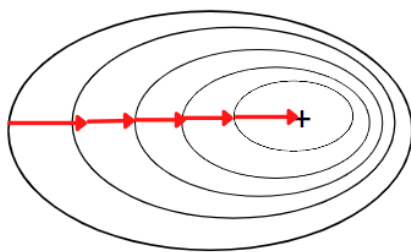
b) Stochastic Gradient Descent (SGD)

- Uses a single data point (example) to compute the gradient.
- Pros: Faster updates, suitable for large datasets.
- Cons: Noisy updates can lead to fluctuation around the minimum.

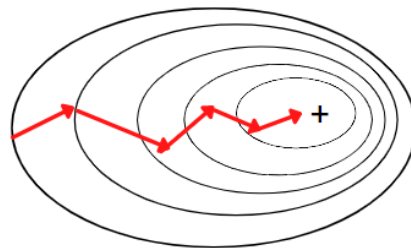
c) Mini-Batch Gradient Descent

- Uses a small batch of data (a subset of the dataset) to compute the gradient.
- Pros: Balances between batch and stochastic methods, commonly used in practice.
- Cons: Still computationally demanding but efficient.

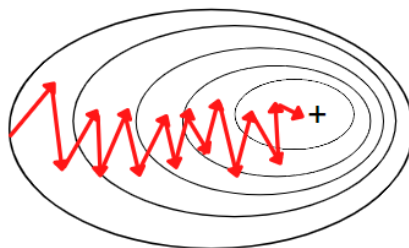
Batch Gradient Descent



Mini-Batch Gradient Descent



Stochastic Gradient Descent



Supervised Learning -Classification

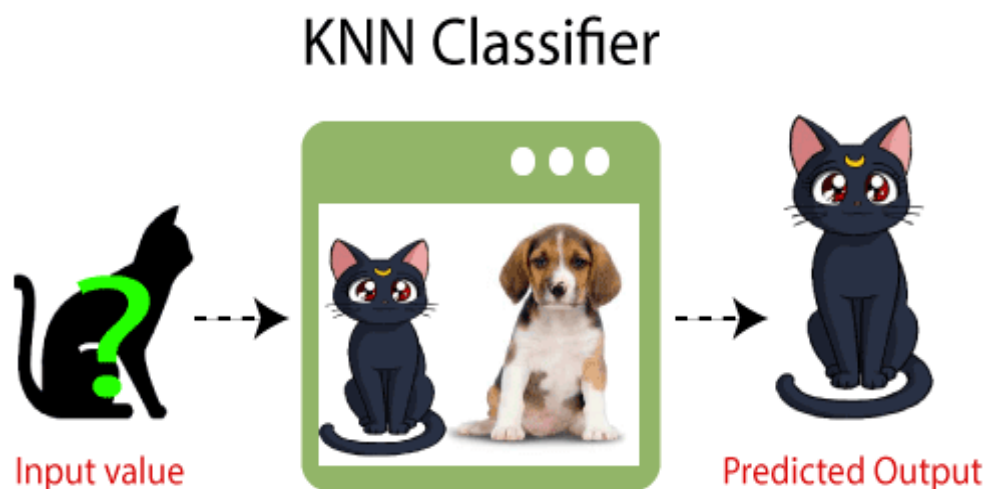
Unit-4

Session-5

K-Nearest Neighbors (KNN) is a **supervised machine learning algorithm** used for classification and regression tasks but mostly it is used for the Classification problems.

K-NN is a non-parametric algorithm, which means it does not make any assumption on underlying data.

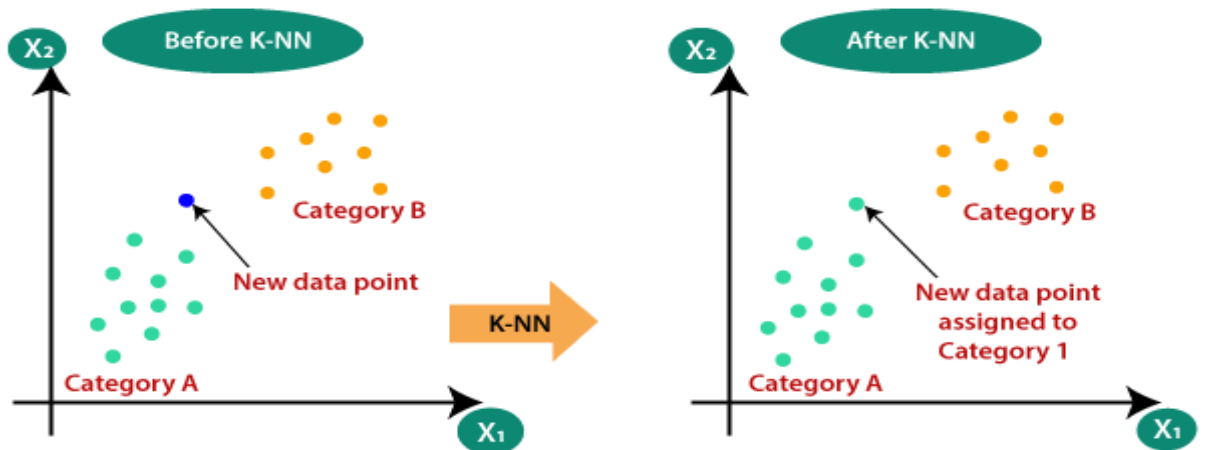
It is also called a lazy learner algorithm because it does not learn from the training set immediately instead it stores the dataset and at the time of classification, it performs an action on the dataset.



- Example: Suppose, we have an image of a creature that looks similar to cat and dog, but we want to know either it is a cat or dog. So for this identification, we can use the KNN algorithm, as it works on a similarity measure. Our KNN model will find the similar features of the new data set to the cats and dogs images and based on the most similar features it will put it in either cat or dog category.

Why do we need a K-NN Algorithm?

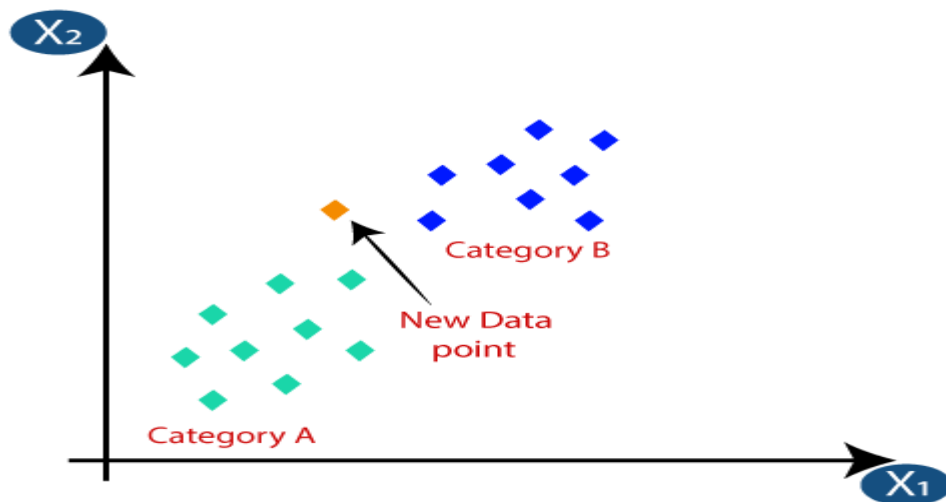
- Suppose there are two categories, i.e., Category A and Category B, and we have a new data point x_1 , so this data point will lie in which of these categories. To solve this type of problem, we need a K-NN algorithm. With the help of K-NN, we can easily identify the category or class of a particular dataset. Consider the below diagram:



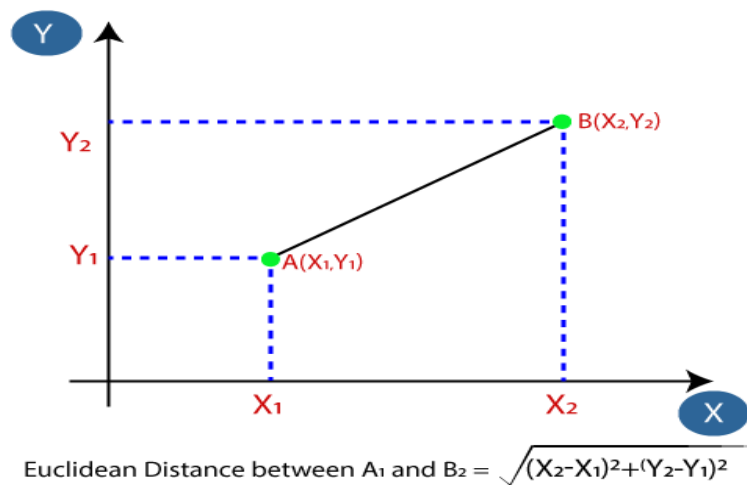
How does K-NN work?

The K-NN working can be explained on the basis of the below algorithm:

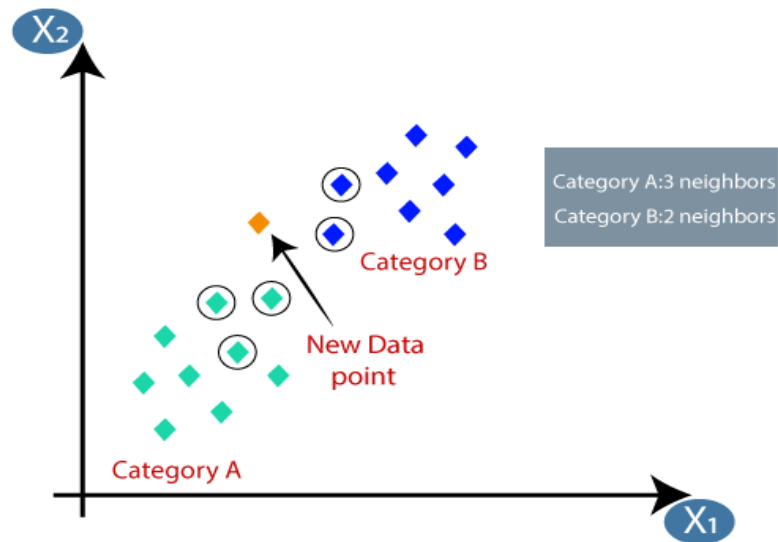
- Step-1: Select the number K of the neighbors
- Step-2: Calculate the Euclidean distance of K number of neighbors
- Step-3: Take the K nearest neighbors as per the calculated Euclidean distance.
- Step-4: Among these k neighbors, count the number of the data points in each category.
- Step-5: Assign the new data points to that category for which the number of the neighbor is maximum.
- Step-6: Our model is ready.



- Firstly, we will choose the number of neighbors, so we will choose the $k=5$.
- Next, we will calculate the Euclidean distance between the data points.
- The Euclidean distance is the distance between two points, which we have already studied in geometry. It can be calculated as:



By calculating the Euclidean distance we got the nearest neighbors, as three nearest neighbors in category A and two nearest neighbors in category B. Consider the below image



As we can see the 3 nearest neighbors are from category A, hence this new data point must belong to category A.

How to select the value of K in the K-NN Algorithm?

Below are some points to remember while selecting the value of K in the K-NN algorithm:

- There is no particular way to determine the best value for "K", so we need to try some values to find the best out of them. The most preferred value for K is 5.
- A very low value for K such as K=1 or K=2, can be noisy and lead to the effects of outliers in the model.
- Large values for K are good, but it may find some difficulties.

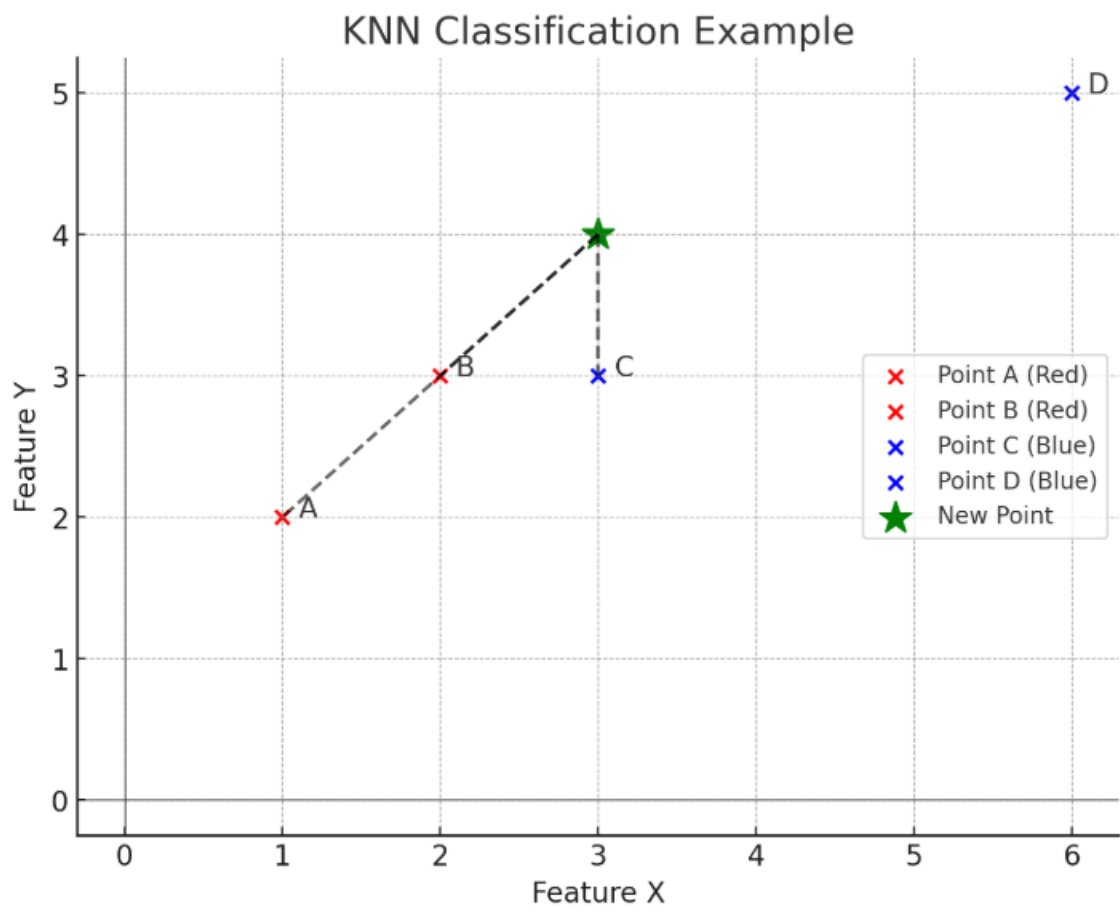
Example: Classification with KNN

- We want to classify a new data point into one of two classes: *Red* or *Blue*.

Dataset

Data Point	Feature X	Feature Y	Class
A	1	2	Red
B	2	3	Red
C	3	3	Blue
D	6	5	Blue

New Data Point: $(x = 3, y = 4)$



1. **Calculate Distances:**

- Using **Euclidean distance:**

- Distance to A: $\sqrt{(3 - 1)^2 + (4 - 2)^2} = \sqrt{4 + 4} = \sqrt{8} = 2.83$
- Distance to B: $\sqrt{(3 - 2)^2 + (4 - 3)^2} = \sqrt{1 + 1} = \sqrt{2} = 1.41$
- Distance to C: $\sqrt{(3 - 3)^2 + (4 - 3)^2} = \sqrt{0 + 1} = 1$
- Distance to D: $\sqrt{(3 - 6)^2 + (4 - 5)^2} = \sqrt{9 + 1} = \sqrt{10} = 3.16$

2. **Select $k = 3$ Neighbors:**

- Closest points: C (1), B (1.41), and A (2.83)

3. **Vote:**

- Classes: C (Blue), B (Red), A (Red)
- Majority class: **Red**

4. **Predict:** The new point is classified as **Red**.

Advantages of KNN

1. Simple and Intuitive: Easy to implement and understand.
2. Non-parametric: Makes no assumptions about data distribution.
3. Versatile: Works for both classification and regression.

Disadvantages

1. Computationally Expensive: Distance calculations for all training points can be slow for large datasets.
2. Memory-intensive: Requires storing the entire training dataset.
3. Sensitive to Irrelevant Features: Can be affected by noise and irrelevant features.
4. Choice of k: Selection of k can significantly affect performance

Applications

- Handwritten digit recognition
- Recommender systems
- Disease classification in healthcare
- Fraud detection in banking

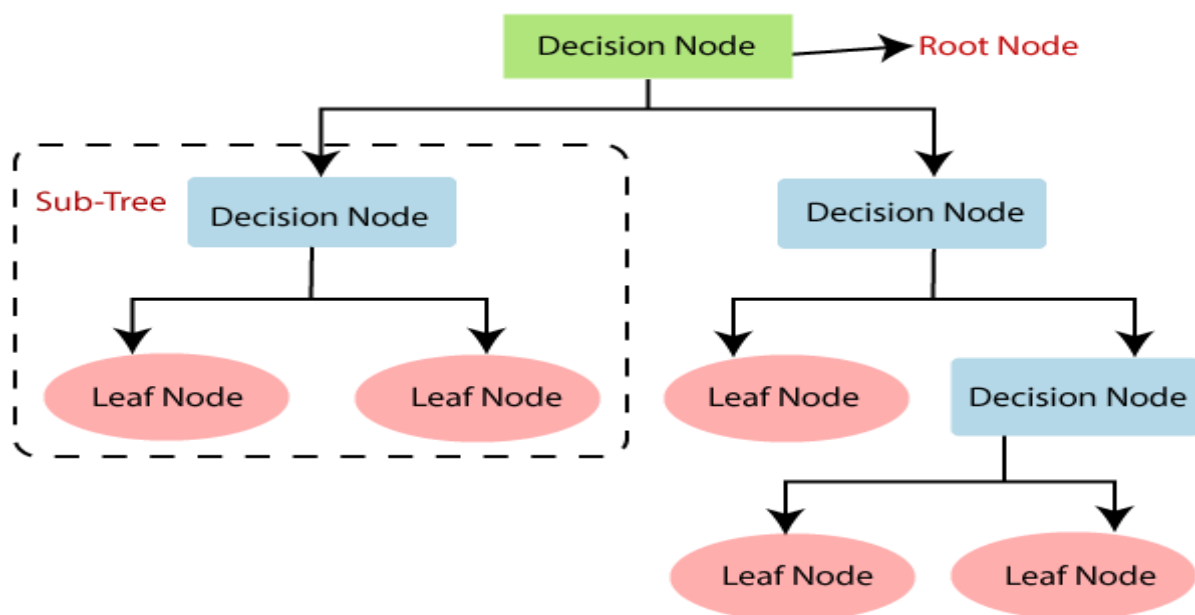
Supervised Learning -Classification

Unit-4

Session-6

Classification Algorithm-Decision trees

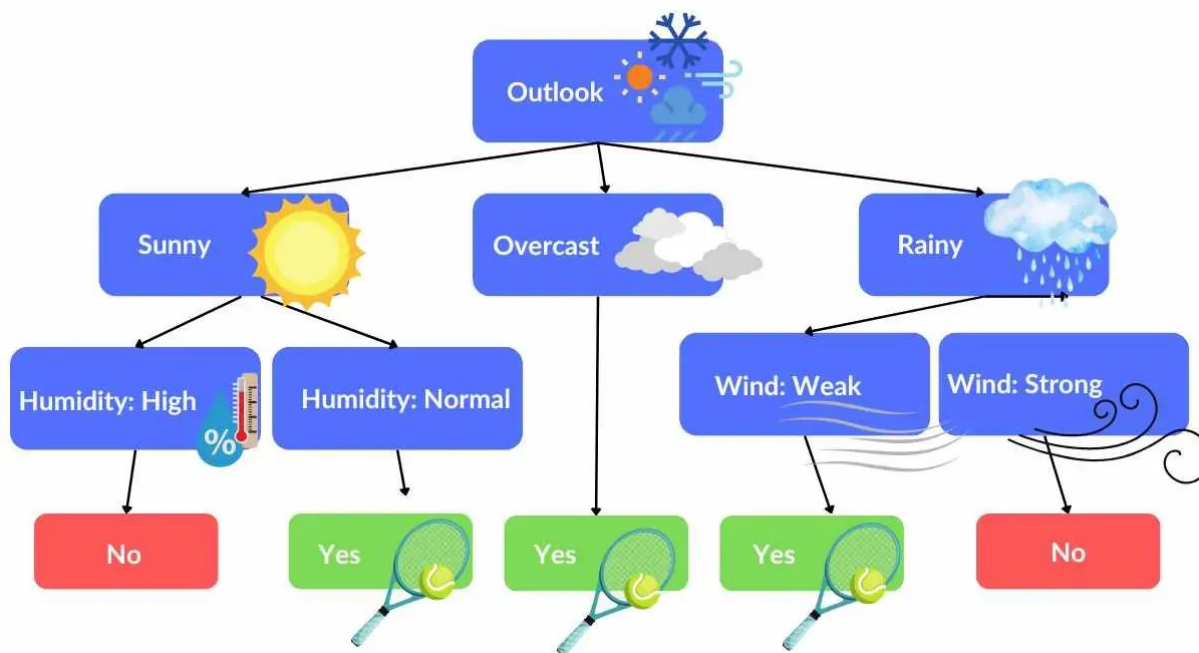
A **Decision Tree** is a supervised machine learning algorithm used for classification and regression tasks. It is structured like a flowchart, where each internal node represents a decision based on a feature, each branch represents an outcome of the decision, and each leaf node represents a class label (classification) or a value (regression).

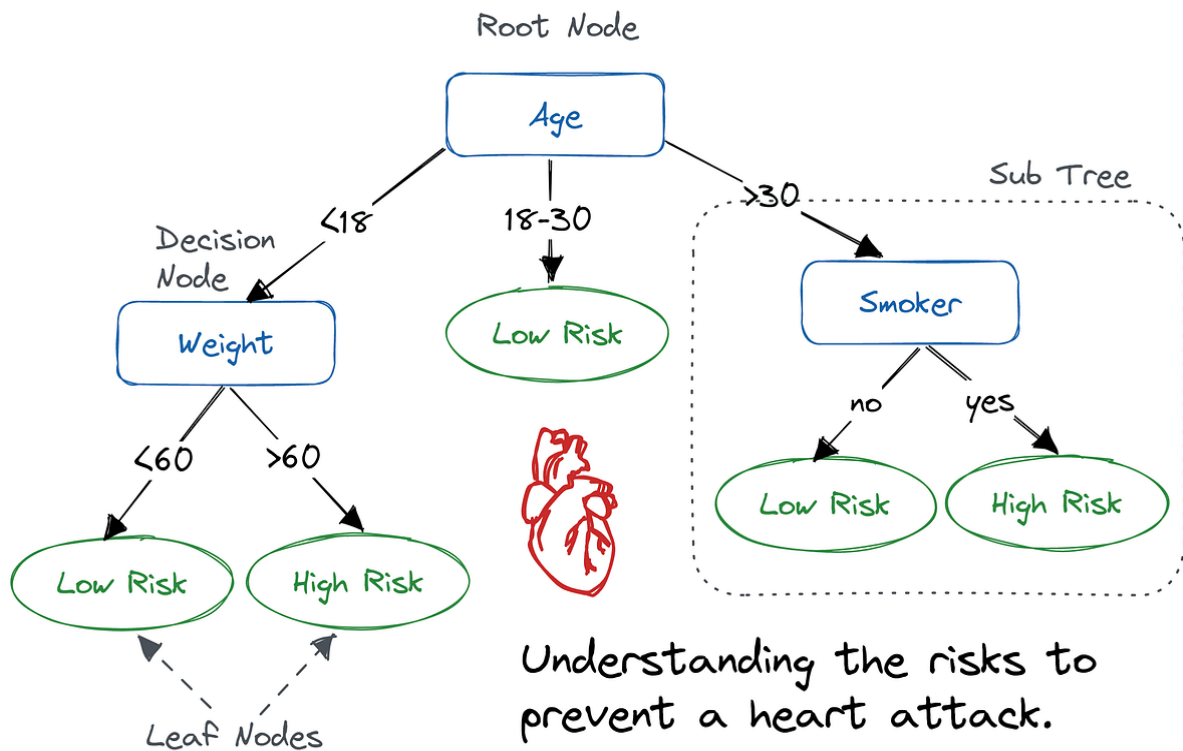


Structure of a Decision Tree

1. **Root Node:**
 1. The starting point of the tree.
 2. Represents the entire dataset and splits it based on the best decision rule.
2. **Decision Nodes:**
 1. Intermediate nodes where features are evaluated, and the dataset is split into subsets.
 2. Each split corresponds to a condition on a feature.
3. **Leaf Nodes:**
 1. Terminal nodes that provide the output (class label or value).
4. **Branches:**
 1. Pathways connecting nodes, representing the decisions or outcomes of a condition.

Day	Outlook	Temp	Humidity	Wind	Tennis
1	 Sunny	Hot	 High	 Weak	No
2	 Sunny	Hot	 High	 Strong	No
3	 Overcast	Hot	 High	 Weak	Yes
4	 Rain	Mild	 High	 Weak	Yes
5	 Rain	Cool	 Normal	 Weak	Yes
6	 Rain	Cool	 Normal	 Strong	No
7	 Overcast	Cool	 Normal	 Weak	Yes





How Decision Trees Work

1. **Select the Best Feature to Split:**
 1. Use a splitting criterion like **Gini Index, Information Gain**.
 2. Choose the feature that best separates the data into pure subsets.
2. **Split the Dataset:**
 1. Divide the dataset based on the selected feature and its values.
3. **Repeat Recursively:**
 1. For each subset, repeat the process until:
 1. The tree reaches a maximum depth.
 2. The dataset in a node is "pure" (only one class remains).
 3. A minimum number of samples per leaf node is reached.
4. **Output:**
 1. For classification: Assign the majority class in the leaf node.
 2. For regression: Assign the average value in the leaf node.

Splitting Criteria

1. Gini Index (Used in CART):

- Measures impurity of a dataset.

Formula:

$$Gini = 1 - \sum_{i=1}^n (p_i)^2$$

- where p_i is the proportion of samples belonging to class i .

2. Information Gain (Used in ID3):

- Measures the reduction in entropy after a split.

Formula:

$$IG = Entropy_{parent} - \sum_{children} \left(\frac{\text{samples in child}}{\text{samples in parent}} \times Entropy_{child} \right)$$

Advantages of Decision Trees

1. **Easy to Understand:** Visual representation is intuitive and interpretable.
2. **Handles Non-linear Relationships:** Can model complex decision boundaries.
3. **No Assumptions About Data:** Works well for both categorical and numerical data.
4. **Feature Importance:** Identifies the most important features.

Disadvantages

1. **Overfitting:** Deep trees can model noise, reducing generalization.
2. **Instability:** Small changes in data can lead to a completely different tree.
3. **Bias:** Trees tend to favor features with more splits (numerical features).

Example: Classification with Decision Tree

4. **Scenario**
5. We want to classify fruits based on their attributes.

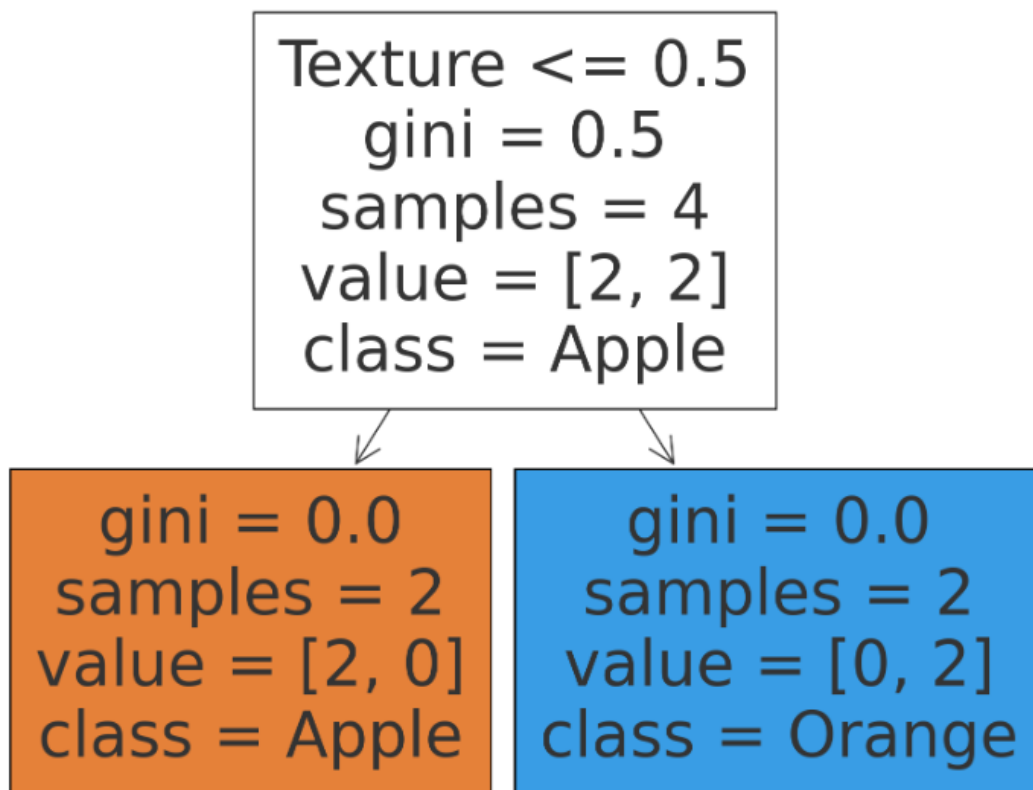
Dataset

Color	Weight (g)	Texture	Fruit
Green	150	Smooth	Apple
Yellow	120	Smooth	Apple
Red	180	Rough	Orange
Yellow	200	Rough	Orange

Tree Construction

1. **Root Node:** Split based on "Color".
2. **Decision Nodes:** Further split based on "Weight" or "Texture".
3. **Leaf Nodes:** Classify as Apple or Orange.

Decision Tree Visualization



Types of Decision Trees

- **ID3:** Ross Quinlan is credited within the development of ID3, which is shorthand for "Iterative Dichotomiser 3." This algorithm leverages entropy and information gain as metrics to evaluate candidate splits.

- **C4.5:** This algorithm is considered a later iteration of ID3, which was also developed by Quinlan. It can use information gain or gain ratios to evaluate split points within the decision trees.

- **CART:** The term, CART, is an abbreviation for “classification and regression trees” and was introduced by Leo Breiman. This algorithm typically utilizes Gini impurity to identify the ideal attribute to split on. Gini impurity measures how often a randomly chosen attribute is misclassified. When evaluating using Gini impurity, a lower value is more ideal.

Supervised Learning -Classification

Unit-4

Session-7

Classification Algorithm: Naïve bayes Classifier

The Naive Bayes classifier is a probabilistic machine learning algorithm based on **Bayes' Theorem**. It is particularly suited for classification problems and is widely used in text classification (e.g., spam detection, sentiment analysis) and other areas requiring a simple yet effective classifier.

"Naive" Assumption

The term "naive" comes from the assumption that all features are **independent** given the class label. In reality, this assumption is often not true, but the classifier still works well in many applications.

1. Bayes' Theorem

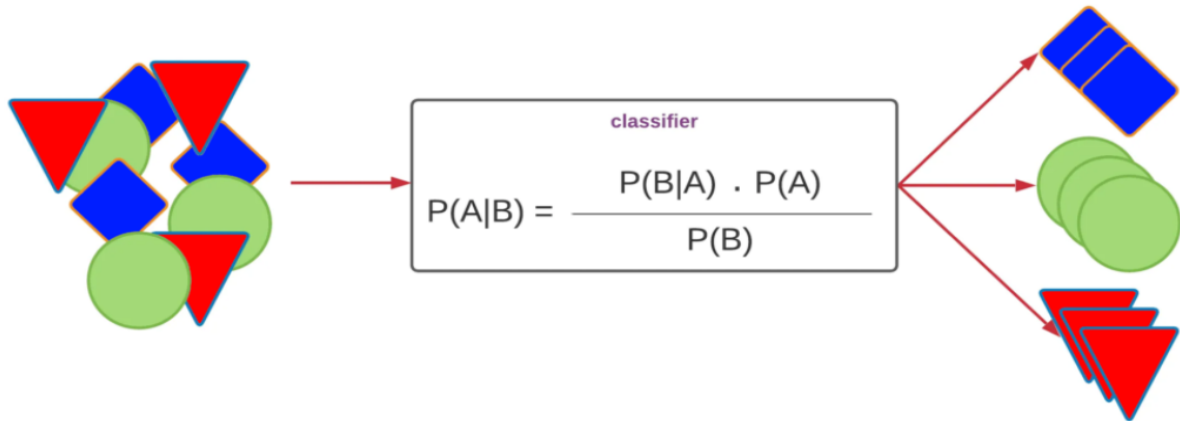
Bayes' Theorem provides a way to calculate the probability of a hypothesis (class) based on prior knowledge and evidence:

$$P(C|X) = \frac{P(X|C) \cdot P(C)}{P(X)}$$

Where:

- $P(C|X)$: Posterior probability — the probability of class C given the features X.
- $P(X|C)$: Likelihood — the probability of features X given the class C.
- $P(C)$: Prior probability — the initial probability of class C (before seeing the features).
- $P(X)$: Marginal probability — the total probability of the features X.

Naive Bayes Classifier



How Naive Bayes Works

The classifier uses the probabilities from Bayes' Theorem to predict the most likely class for a given instance. Here's the step-by-step process:

Training Phase:

- Compute the **prior probability** $P(C)$ for each class C from the training data.
- Compute the **likelihood** $P(X|C)$ for each feature X given the class C .
- If the data is continuous, probabilities are computed using distributions like Gaussian. For categorical data, probabilities are calculated as frequencies.

$$P(C|X) \propto P(C) \prod_{i=1}^n P(x_i|C)$$

Prediction Phase:

- Given a new instance $X=\{x_1, x_2, \dots, x_n\} = \{x_1, x_2, \dots, x_n\}$, calculate the posterior probability for each class C :

$$\text{Predicted Class} = \arg \max_C P(C|X)$$

- Assign the class with the highest posterior probability:

Types of Naive Bayes Classifiers

1. Gaussian Naive Bayes:

1. Assumes the data follows a Gaussian (normal) distribution.

2. Used for continuous data.
2. **Multinomial Naive Bayes:**
 1. Assumes feature frequencies are drawn from a multinomial distribution.
 2. Commonly used for document classification (e.g., bag-of-words model).
3. **Bernoulli Naive Bayes:**
 1. Features are binary (presence/absence of a feature).
 2. Suitable for binary data.

Example: Spam Email Classification

1. **Training:**
 1. Classify emails into "Spam" or "Not Spam".
 2. Compute $P(\text{Spam})$ and $P(\text{NotSpam})$.
 3. Calculate the likelihood $P(\text{word}|\text{Spam})$ and $P(\text{word}|\text{NotSpam})$ for all words.
2. **Prediction:**
 1. For a new email, calculate $P(\text{Spam}|\text{Email})$ and $P(\text{NotSpam}|\text{Email})$.
 2. Classify the email based on the higher posterior probability.
3. The Naive Bayes consists of two words: 1- Naive: As it assumes the independency between traits or features. 2- Bayes: Based on Bayes' theorem.
4. To use the algorithm:
 - 1- We must convert the presented data set into frequency tables
 - 2- Then create a probability table by finding the probabilities of certain features.
 - 3- Then use Bayes' theorem in order to calculate the posterior probability.

Given the following dataset:

- For example, let's solve the following problem: If the weather is sunny, then the Player should play or not?

	Outlook	Play
0	Rainy	Yes
1	Sunny	Yes
2	Overcast	Yes
3	Overcast	Yes
4	Sunny	No
5	Rainy	Yes
6	Sunny	Yes
7	Overcast	Yes
8	Rainy	No
9	Sunny	No
10	Sunny	Yes
11	Rainy	No
12	Overcast	Yes
13	Overcast	Yes

- The first step is to convert our data into frequency tables as follows:

Weather	No	Yes	
Overcast	0	5	5
Rainy	2	2	4
Sunny	2	3	5
All	4	10	14

Convert The Dataset to Frequency table.

- Then to create the likelihood/probability table as follows:

Weather	No	Yes	
Overcast	0	5	5/14=0.35
Rainy	2	2	4/14=0.29
Sunny	2	3	5/14=0.35
All	4/14=0.29	10/14=0.71	

Likelihood/probability table

Now let's apply the algorithm to our case:

```

P(Yes|Sunny) = P(Sunny|Yes)*P(Yes)/P(Sunny)
P(Sunny|Yes) = 3/10 = 0.3
P(Sunny) = 0.35
P(Yes) = 0.71
P(Yes|Sunny) = 0.3*0.71/0.35 = 0.6
P(No|Sunny) = P(Sunny|No)*P(No)/P(No)
P(Sunny|No) = 2/4 = 0.5
P(Sunny) = 0.35
P(No) = 0.29
P(No|Sunny) = 0.5*0.29/0.35 = 0.41

```

Naive Bayes Algorithm Calculations

$P(Yes|Sunny) > P(No|Sunny) \Rightarrow$ So on a sunny day, the player can play the game.

Advantages

1. **Simple and Fast:** Easy to implement and computationally efficient.
2. **Scalable:** Handles large datasets well.
3. **Performs Well with Small Data:** Works well even with limited training data.
4. **Handles High-Dimensional Data:** Often used for text classification with thousands of features.

Disadvantages

1. **Independence Assumption:** Assumes features are independent, which is rarely true in real-world data.
2. **Continuous Variables:** May not perform well if the data distribution doesn't match the assumed Gaussian distribution in the case of Gaussian Naive Bayes.

Applications

- Text classification (e.g., sentiment analysis, spam detection).
- Medical diagnosis.
- Recommendation systems.
- Face recognition.

Supervised Learning -Classification

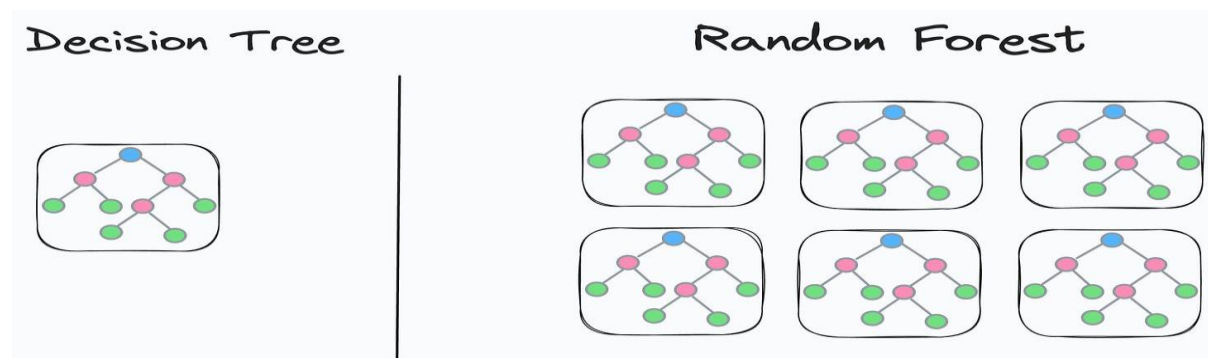
Unit-4

Session-8

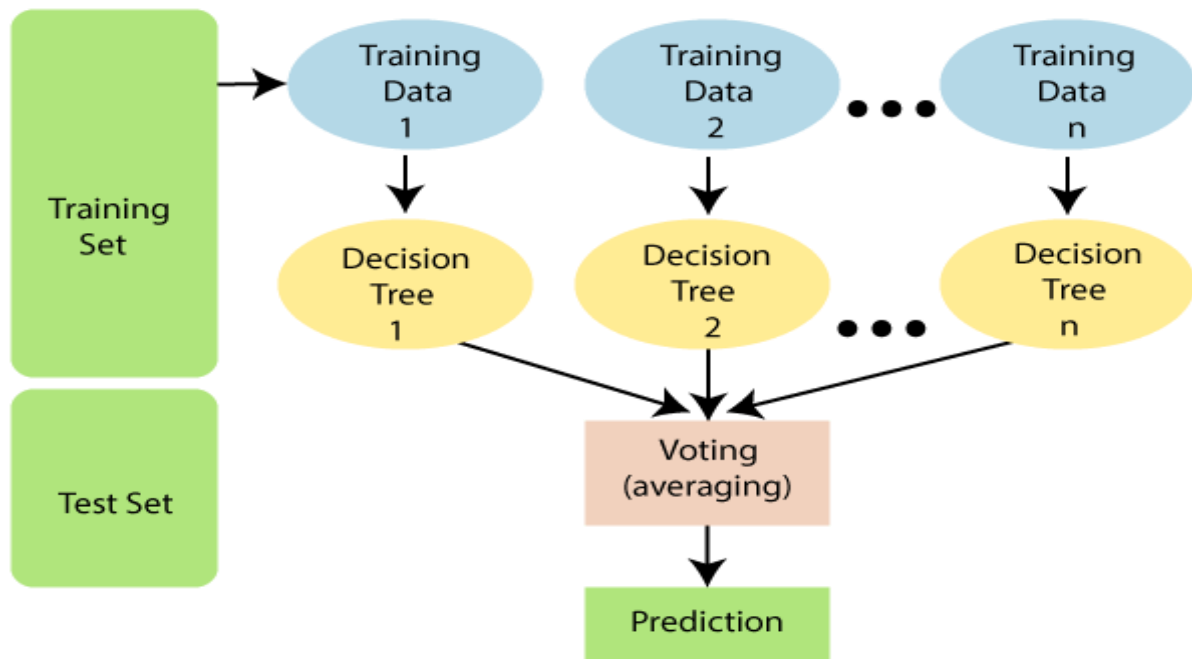
Classification Algorithm: Random Forest

What is the Random Forest Algorithm?

Random Forest algorithm is a powerful tree learning technique in Machine Learning. It works by creating a number of Decision Trees during the training phase. Each tree is constructed using a random subset of the data set to measure a random subset of features in each partition. This randomness introduces variability among individual trees, reducing the risk of overfitting and improving overall prediction performance.



In prediction, the algorithm aggregates the results of all trees, either by voting (for classification tasks) or by averaging (for regression tasks). This collaborative decision-making process, supported by multiple trees with their insights, provides an example of stable and precise results. Random forests are widely used for classification and regression functions, which are known for their ability to handle complex data, reduce overfitting, and provide reliable forecasts in different environments.



What are Ensemble Learning models?

Ensemble learning models work just like a group of diverse experts teaming up to make decisions – think of them as a bunch of friends with different strengths tackling a problem together.

Similarly, in ensemble learning, different models, often of the same type or different types, team up to enhance predictive performance. It's all about leveraging the collective wisdom of the group to overcome individual limitations and make more informed decisions in various machine learning tasks. Some popular ensemble models include- XGBoost, AdaBoost, LightGBM, Random Forest, Bagging, Voting etc.

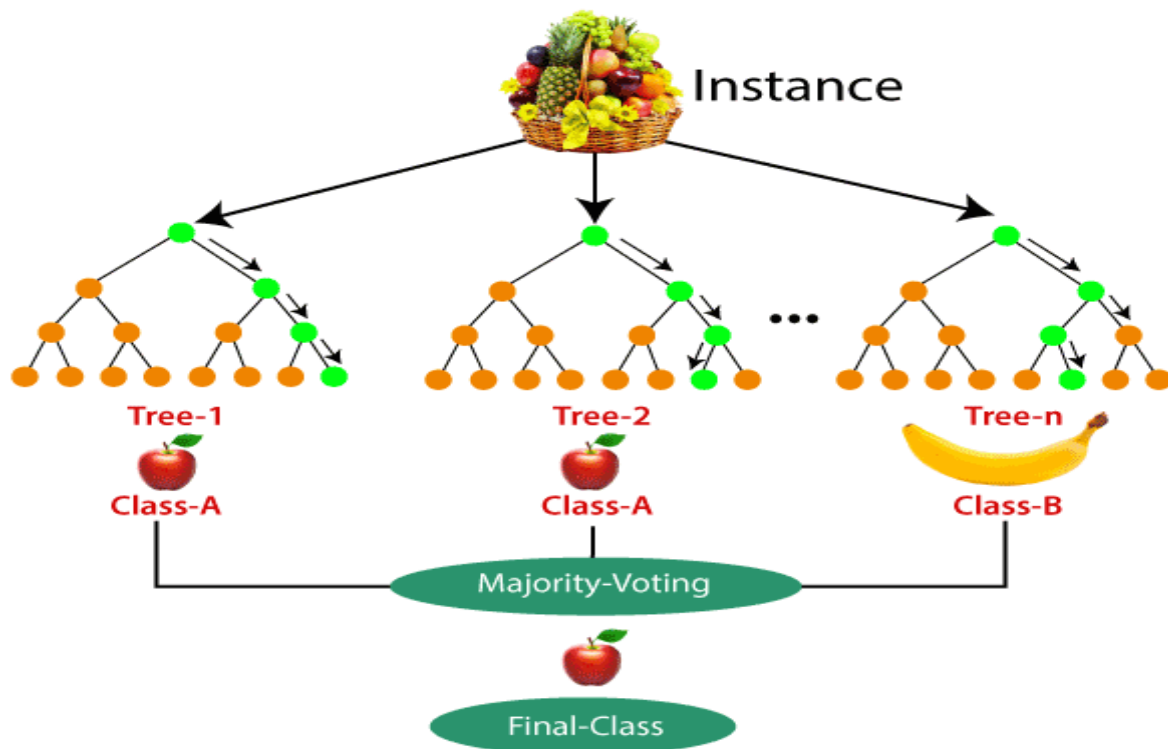
What is Bagging and Boosting?

Bagging is an ensemble learning model, where multiple weak models are trained on different subsets of the training data. Each subset is sampled with replacement and prediction is made by averaging the prediction of the weak models for regression problem and considering majority vote for classification problem.

Boosting trains multiple based models sequentially. In this method, each model tries to correct the errors made by the previous models. Each model is trained on a modified version of the dataset, the instances that were misclassified by the previous models are given more weight. The final prediction is made by weighted voting.

How Does Random Forest Work?

- **Ensemble of Decision Trees:** Random Forest leverages the power of ensemble learning by constructing an army of Decision Trees. These trees are like individual experts, each specializing in a particular aspect of the data. Importantly, they operate independently, minimizing the risk of the model being overly influenced by the nuances of a single tree.
- **Random Feature Selection:** To ensure that each decision tree in the ensemble brings a unique perspective, Random Forest employs random feature selection. During the training of each tree, a random subset of features is chosen. This randomness ensures that each tree focuses on different aspects of the data, fostering a diverse set of predictors within the ensemble.
- **Bootstrap Aggregating or Bagging:** The technique of bagging is a cornerstone of Random Forest's training strategy which involves creating multiple bootstrap samples from the original dataset, allowing instances to be sampled with replacement. This results in different subsets of data for each decision tree, introducing variability in the training process and making the model more robust.
- **Decision Making and Voting:** When it comes to making predictions, each decision tree in the Random Forest casts its vote. For classification tasks, the final prediction is determined by the mode (most frequent prediction) across all the trees. In regression tasks, the average of the individual tree predictions is taken. This internal voting mechanism ensures a balanced and collective decision-making process.



Key Features of Random Forest

- **Handles Missing Values:** Random Forest can handle missing values effectively by using surrogate splits.
- **Handles High Dimensionality:** It works well even with a large number of features.
- **Feature Importance:** It provides a measure of feature importance, helping to understand which variables contribute most to predictions.
- **Robustness to Noise:** Randomness in data and feature selection reduces the impact of noisy or irrelevant features.
- **Overfitting Prevention:** Ensemble methods and random feature selection reduce overfitting compared to a single decision tree.

Let's consider a **real-world example** where a Random Forest model is used to predict whether a loan applicant will default based on their financial and demographic information. Below is an explanation, followed by a flowchart to visualize the Random Forest process.

Steps in Random Forest Process

1. **Data Preprocessing:**

- Clean the data, handle missing values, and normalize numerical features.

2. **Bootstrapping:**

- Random subsets of the training data are created using sampling with replacement.

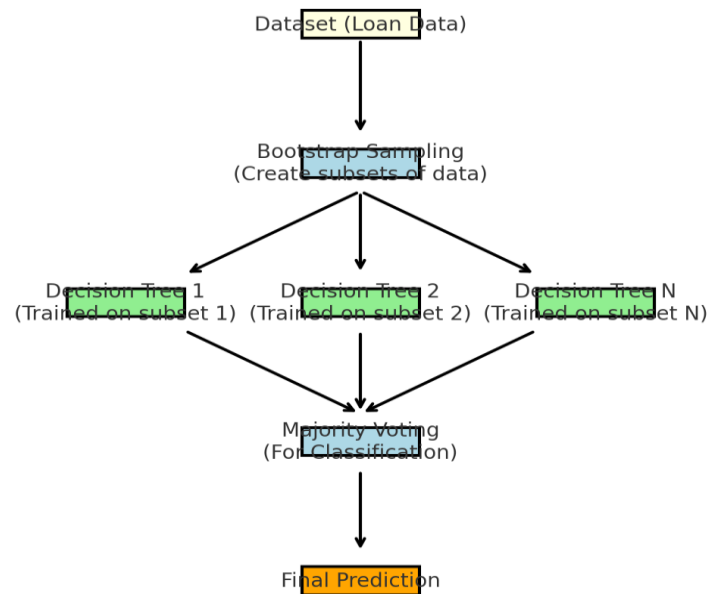
3. **Training Individual Trees:**

- For each bootstrap sample:
 - A decision tree is trained using a random subset of features (e.g., "Income" and "Credit Score").
 - The tree predicts whether an applicant defaults.

4. **Ensemble Prediction:**

- For classification:
 - Each tree in the forest votes for a class (Yes or No).
 - The class with the majority votes is selected.
- For regression:
 - Average the predictions of all trees.

Random Forest Process Flowchart



Dataset Description

We have a dataset with the following features:

1. Age: Age of the applicant.
2. Income: Monthly income of the applicant.
3. Credit Score: The applicant's credit score.
4. Loan Amount: The amount of loan applied for.
5. Loan Tenure: Duration of the loan in years.

The target variable is:

- Loan Default: Whether the applicant defaulted on the loan (Yes or No).

Advantages

- Random Forest is capable of performing both Classification and Regression tasks.
- It is capable of handling large datasets with high dimensionality.
- It enhances the accuracy of the model and prevents the overfitting issue.

Disadvantages

- **Computational Cost:** Training a large number of trees can be computationally expensive and memory-intensive.
- **Interpretability:** While individual trees are interpretable, the overall Random Forest model is more complex and harder to interpret.
- **Bias with Small Data:** It may not perform as well with very small datasets compared to simpler models.

Applications of Random Forest

There are mainly four sectors where Random forest mostly used:

1. **Banking:** Banking sector mostly uses this algorithm for the identification of loan risk.
2. **Medicine:** With the help of this algorithm, disease trends and risks of the disease can be identified.
3. **Land Use:** We can identify the areas of similar land use by this algorithm.
4. **Marketing:** Marketing trends can be identified using this algorithm.

Supervised Learning -Classification

Unit-4

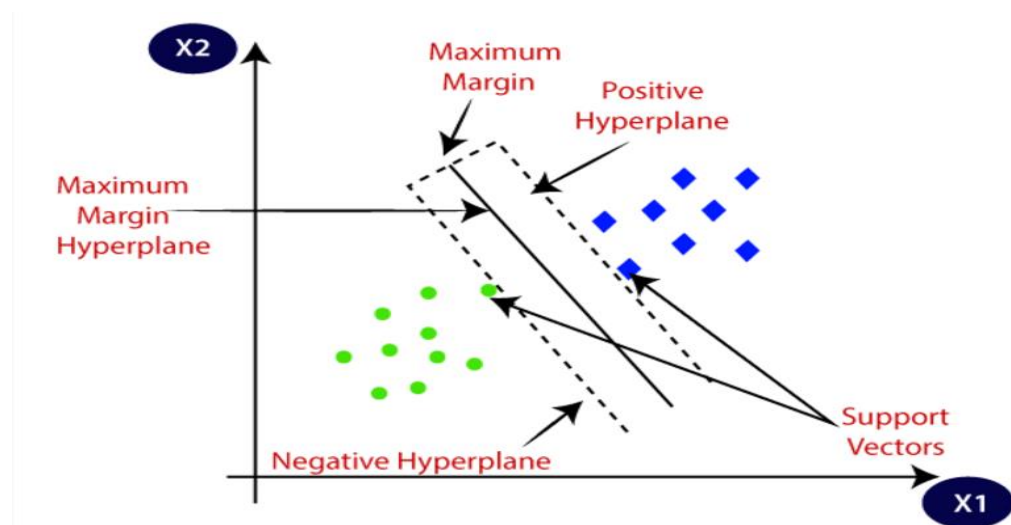
Session-9

Classification Algorithm: Support Vector machine

Support Vector Machine or SVM is one of the most popular Supervised Learning algorithms, which is used for Classification as well as Regression problems. However, primarily, it is used for Classification problems in Machine Learning.

The goal of the SVM algorithm is to create the best line or decision boundary that can segregate n-dimensional space into classes so that we can easily put the new data point in the correct category in the future. This best decision boundary is called a hyperplane.

SVM chooses the extreme points/vectors that help in creating the hyperplane. These extreme cases are called as support vectors, and hence algorithm is termed as Support Vector Machine. Consider the below diagram in which there are two different categories that are classified using a decision boundary or hyperplane:



Types of SVM

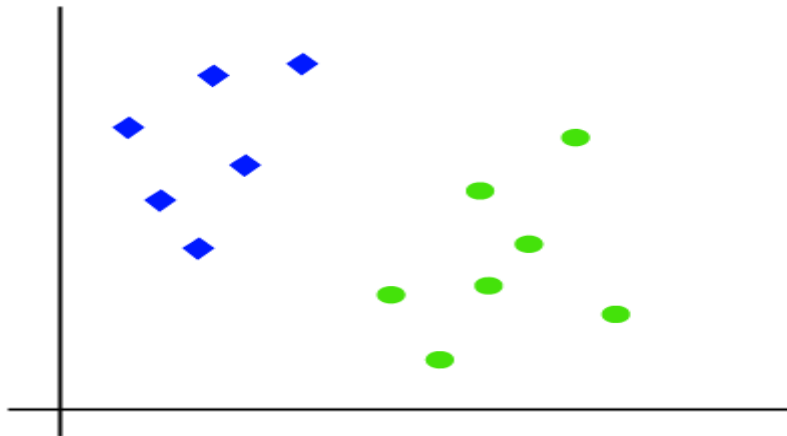
SVM can be of two types:

- **Linear SVM:** Linear SVM is used for linearly separable data, which means if a dataset can be classified into two classes by using a single straight line, then such data is termed as linearly separable data, and classifier is used called as Linear SVM classifier.

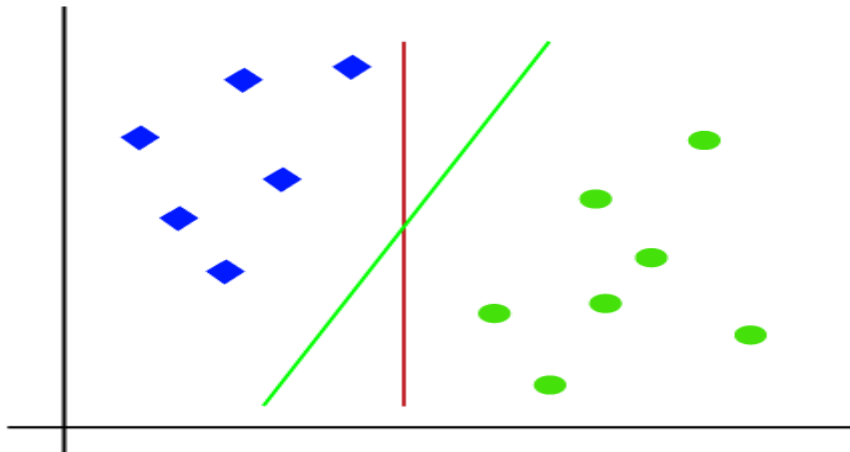
- **Non-linear SVM:** Non-Linear SVM is used for non-linearly separated data, which means if a dataset cannot be classified by using a straight line, then such data is termed as non-linear data and classifier used is called as Non-linear SVM classifier.

How does SVM works?

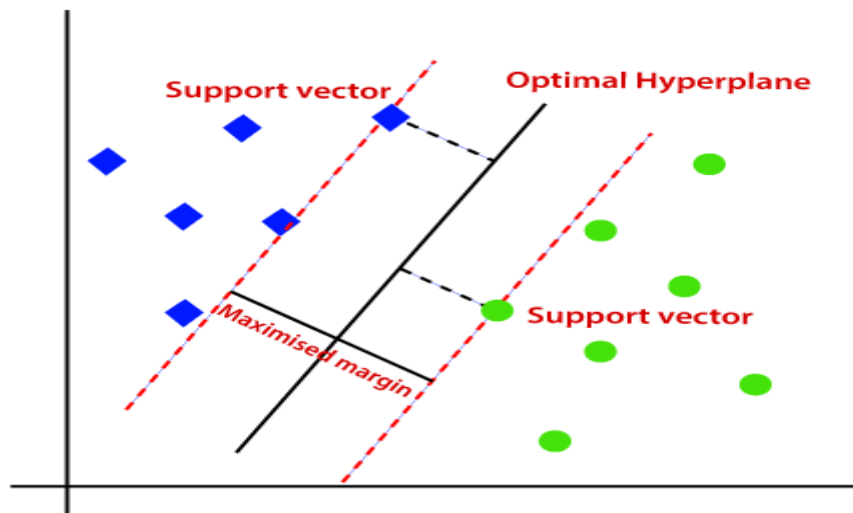
- **Linear SVM:**
- The working of the SVM algorithm can be understood by using an example. Suppose we have a dataset that has two tags (green and blue), and the dataset has two features x_1 and x_2 . We want a classifier that can classify the pair(x_1 , x_2) of coordinates in either green or blue. Consider the below image:



So as it is 2-d space so by just using a straight line, we can easily separate these two classes. But there can be multiple lines that can separate these classes. Consider the below image

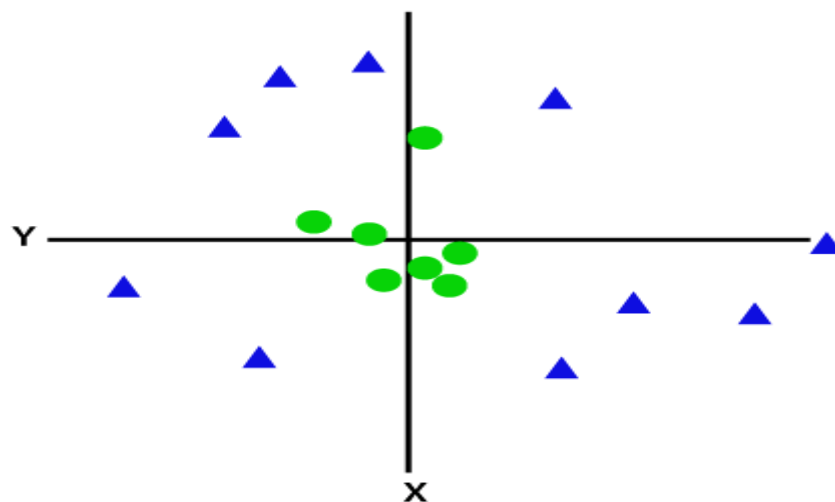


Hence, the SVM algorithm helps to find the best line or decision boundary; this best boundary or region is called as a **hyperplane**. SVM algorithm finds the closest point of the lines from both the classes. These points are called support vectors. The distance between the vectors and the hyperplane is called as **margin**. And the goal of SVM is to maximize this margin. The **hyperplane** with maximum margin is called the **optimal hyperplane**.



Non-Linear SVM:

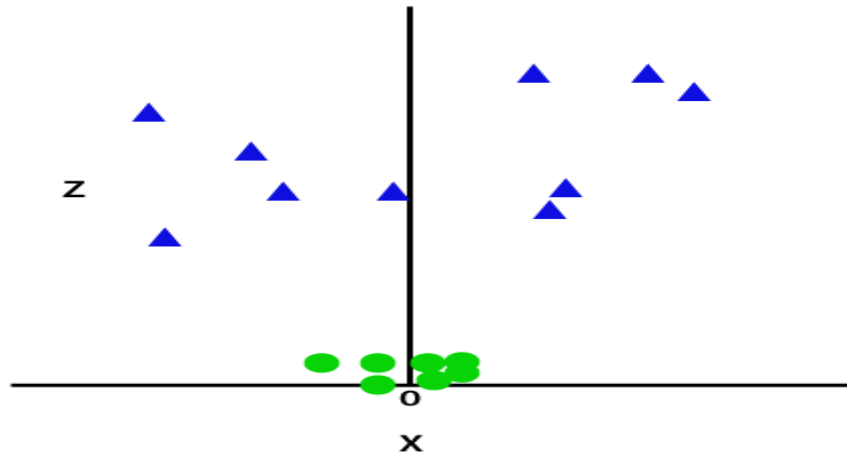
If data is linearly arranged, then we can separate it by using a straight line, but for non-linear data, we cannot draw a single straight line. Consider the below image:



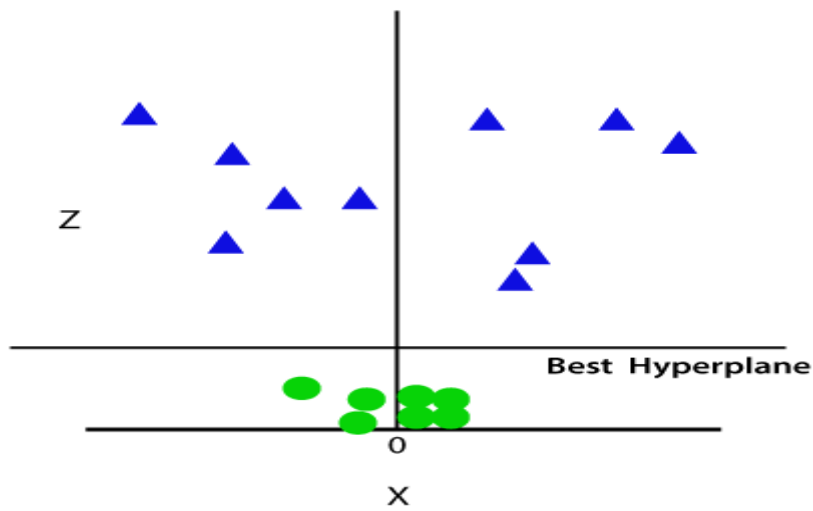
So to separate these data points, we need to add one more dimension. For linear data, we have used two dimensions x and y, so for non-linear data, we will add a third dimension z. It can be calculated as:

$$z = x^2 + y^2$$

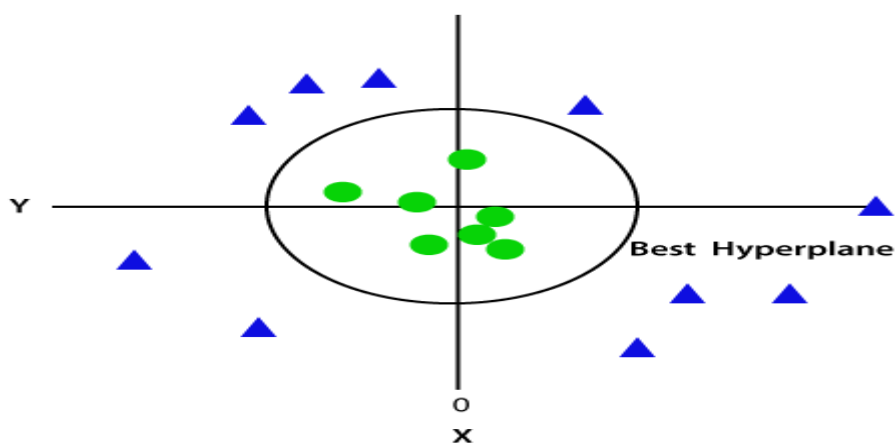
By adding the third dimension, the sample space will become as below image:



So now, SVM will divide the datasets into classes in the following way. Consider the below image:



Since we are in 3-d Space, hence it is looking like a plane parallel to the x-axis. If we convert it in 2d space with $z=1$, then it will become as:



Hence we get a circumference of radius 1 in case of non-linear data.

Dataset Example

- Features:

- Number of spam words (f1)
- Frequency of specific keywords (f2)
- Length of the email (f3)
- Other metrics.
- **Labels:**
 - Spam (1)
 - Not Spam (0)

Training Process

1. Input features and labels into the SVM algorithm.
2. The SVM calculates the hyperplane to separate Spam from Not Spam.
3. Support Vectors (emails close to the boundary) define the margin.

Prediction

Given a new email, SVM calculates its position relative to the hyperplane and predicts whether it's Spam or Not Spam.

Advantages

1. **Effective in High-Dimensional Spaces**
 1. SVM performs well in cases where the number of dimensions (features) is greater than the number of samples.
2. **Works Well for Clear Margin of Separation**
 1. It is highly effective for datasets with a clear boundary between classes.
3. **Versatile with Kernel Functions**
 1. Using different kernel functions, SVM can handle both linear and non-linear classification tasks.
4. **Robust to Overfitting (Especially in High Dimensions)**
 1. SVM maximizes the margin, reducing the chance of overfitting, particularly when the number of features is high.
5. **Effective in Both Classification and Regression Tasks**
 1. While primarily used for classification, it can also perform regression (Support Vector Regression - SVR).

Disadvantages

1. **Computationally Intensive**
 1. Training an SVM can be slow for large datasets, especially with complex kernel functions.

2. Not Ideal for Large Datasets

1. When the number of data points is very large, SVM struggles due to its computational complexity ($O(n^2)$ or more).

3. Requires Careful Selection of Kernel and Parameters

1. The performance depends heavily on choosing the right kernel and tuning parameters like C (regularization) and γ (for RBF kernel).

4. Less Effective with Overlapping Classes

1. SVM struggles when classes are not well-separated or have significant overlap.

Applications

1. Healthcare

- **Disease Diagnosis:** Predicting diseases like cancer (e.g., breast cancer diagnosis using medical imaging data).
- **Genomics:** Classifying gene expression data.

2. Finance

- **Fraud Detection:** Identifying fraudulent transactions in banking.
- **Credit Scoring:** Determining credit risk for loans.

3. Text and Image Recognition

- **Spam Filtering:** Classifying emails as spam or not spam.
- **Handwritten Digit Recognition:** Identifying handwritten characters (e.g., in the MNIST dataset).
- **Face Detection:** Recognizing and classifying faces in images.

EXP 6: Implement a logistic regression model on given dataset and check the accuracy for test dataset.

Logistic Regression is a supervised machine learning algorithm used for binary classification problems. Unlike Linear Regression, it predicts the probability of a target variable belonging to a specific class by applying a logistic (sigmoid) function to model outcomes between 0 and 1.

Step1:Load the Dataset

The Titanic dataset is used for this example. It contains data on passengers, including demographic information (e.g., age, gender) and travel details (e.g., ticket class, fare). The goal is to predict whether a passenger survived or not (Survived column).

The head() function is used to understand the structure and contents of the dataset.

Key Features: The dataset includes variables such as Pclass (ticket class), Sex (gender), and Age, which are vital for survival prediction.

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Step 1: Load the dataset
# Using the Titanic dataset as an example
data = pd.read_csv('https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv')

# Display the first few rows of the dataset
print("Dataset preview:")
print(data.head())
```



Dataset preview:

	PassengerId	Survived	Pclass	\
0	1	0	3	
1	2	1	1	
2	3	1	3	
3	4	1	1	
4	5	0	3	

Name	Sex	Age	SibSp	\
------	-----	-----	-------	---

0		Braund, Mr. Owen Harris	male	22.0	1
1	Cumings, Mrs. John Bradley	(Florence Briggs Th...	female	38.0	1
2		Heikkinen, Miss. Laina	female	26.0	0
3	Futrelle, Mrs. Jacques Heath	(Lily May Peel)	female	35.0	1
4		Allen, Mr. William Henry	male	35.0	0

	Parch	Ticket	Fare	Cabin	Embarked
0	0	A/5 21171	7.2500	NaN	S
1	0	PC 17599	71.2833	C85	C
2	0	STON/O2. 3101282	7.9250	NaN	S
3	0	113803	53.1000	C123	S
4	0	373450	8.0500	NaN	S

Step2. Preprocessing

Data preprocessing prepares the dataset for analysis:

Feature Selection: Relevant features (Pclass, Sex, Age, etc.) are chosen based on their importance in predicting survival.

Encoding Categorical Variables: The Sex column is converted into numeric values (0 for male, 1 for female).

Handling Missing Data: Rows with missing values in any of the selected features are removed to ensure consistency.

The dataset is then split into training and testing subsets (80%-20% split), with the training data used to fit the model and the testing data reserved for evaluation.

```
# Step 2: Preprocess the data
# Selecting relevant features and target
selected_features = ['Pclass', 'Sex', 'Age', 'SibSp', 'Parch', 'Fare']
data = data[selected_features + ['Survived']]
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1}) # Encode categorical variable

data = data.dropna() # Remove rows with missing values

X = data[selected_features] # Features
y = data['Survived']       # Target

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

 <ipython-input-11-20f97133c5e4>:5: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.

Try using `.loc[row_indexer,col_indexer] = value` instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1}) # Encode categorical variable
```

Step3. Model Training

A Logistic Regression model is instantiated and trained using the `LogisticRegression` class from `sklearn`. The algorithm learns the relationship between the features (`X_train`) and the target (`y_train`).

```
# Step 3: Initialize and train the Logistic Regression model
model = LogisticRegression()
model.fit(X_train, y_train)
```



► `LogisticRegression` ⓘ ?

Step 4. Making Predictions

The model predicts outcomes (`y_pred`) for the test dataset (`X_test`). These predictions indicate whether a passenger is likely to survive (1) or not (0).

```
# Step 4: Make predictions
y_pred = model.predict(X_test)

# Display predictions for the first 10 test samples
print("\nSample predictions:")
print(f"Predicted: {y_pred[:10]}")
print(f"Actual:     {y_test.values[:10]}")
```



```
Sample predictions:
Predicted: [0 0 1 1 0 0 0 0 1 1]
Actual:    [0 1 1 1 0 1 1 1 0 0]
```

Step 5. Model Evaluation

Accuracy: Measures the percentage of correctly predicted labels in the test set.

Classification Report: Provides precision, recall, F1-score, and support for each class, offering insights into the model's performance.

Confusion Matrix: Visualizes the performance of the classification model, showing the counts of true positives, true negatives, false positives, and false negatives.

Predicted

0

1

0

TN

0 is predicted as 0

FP

0 is predicted as 1

Actual

1

FN

1 is predicted as 0

TP

1 is predicted as 1

$$\text{specificity} = \frac{\text{TN}}{\text{TN} + \text{FP}}$$

$$\text{sensitivity} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

$$\text{accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

```
# Step 5: Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
print(f"\nAccuracy on test data: {accuracy * 100:.2f}%")
```

```
# Additional performance metrics
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```



Accuracy on test data: 74.83%

Classification Report:

	precision	recall	f1-score	support
0	0.78	0.82	0.80	87
1	0.69	0.64	0.67	56
accuracy			0.75	143
macro avg	0.74	0.73	0.73	143
weighted avg	0.75	0.75	0.75	143

STEP 6. Visualizations

Confusion Matrix Heatmap: Displays the confusion matrix using a heatmap for an intuitive understanding of classification errors.

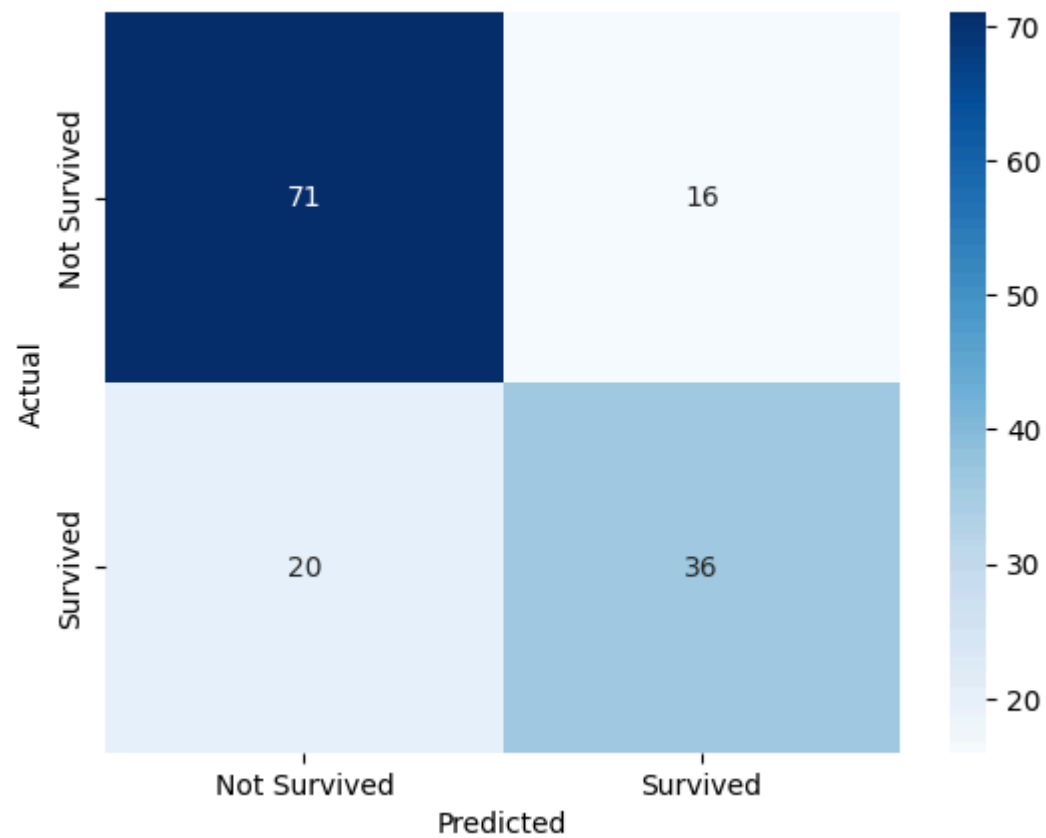
Prediction Distribution: A count plot of predicted classes illustrates the distribution of predictions (survived vs. not survived).

```
# Visualizations
# Confusion Matrix
conf_matrix = confusion_matrix(y_test, y_pred)
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues', xticklabels=['Not Survived', 'Survived'], yticklabels=['Not Survived', 'Survived'])
plt.title('Confusion Matrix')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()

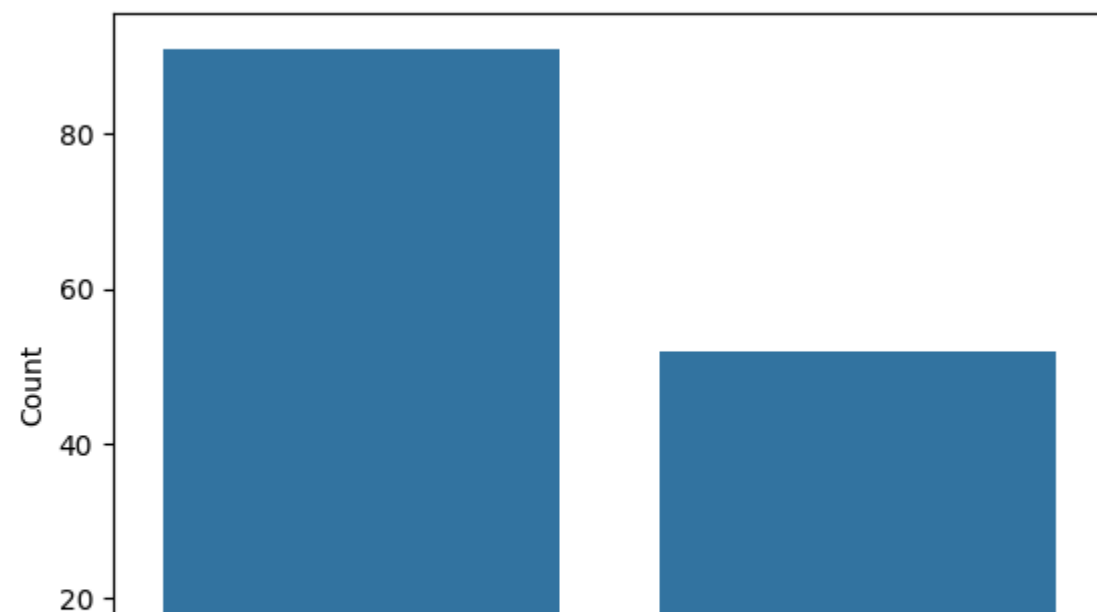
# Distribution of predictions
sns.countplot(x=y_pred)
plt.title('Distribution of Predictions')
plt.xlabel('Predicted Class')
plt.ylabel('Count')
plt.show()
```



Confusion Matrix



Distribution of Predictions





EXP 7: Build a Decision tree model for given dataset to predict the target with best accuracy.

Decision Tree

A Decision Tree is a supervised learning algorithm used for classification and regression tasks. It works by splitting the data into subsets based on feature values, creating a tree structure where each node represents a decision rule, and each leaf node represents a target class or value.

Step 1. Load the Dataset

Use a publicly available dataset or custom data.

Preview the data to understand its structure and key features.

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Step 1: Load the dataset
data = pd.read_csv('https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv')

# Display the first few rows of the dataset
print("Dataset preview:")
print(data.head())
```

↗ Dataset preview:

	PassengerId	Survived	Pclass	\	
0	1	0	3		
1	2	1	1		
2	3	1	3		
3	4	1	1		
4	5	0	3		

	Name	Sex	Age	SibSp	\	
0	Braund, Mr. Owen Harris	male	22.0	1		
1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1		
2	Heikkinen, Miss. Laina	female	26.0	0		
3	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1		
4	Allen, Mr. William Henry	male	35.0	0		

	Parch	Ticket	Fare	Cabin	Embarked
0	0	A/5 21171	7.2500	NaN	S
1	0	PC 17599	71.2833	C85	C
2	0	STON/O2. 3101282	7.9250	NaN	S
3	0	113803	53.1000	C123	S
4	0	373450	8.0500	NaN	S

Step 2: Preprocess the Data

Feature Selection: Identify independent variables and the target variable.

Handle Missing Values: Replace or remove missing data to ensure consistency.

Encode Categorical Variables: Convert non-numeric data into numeric form for processing.

```
# Step 2: Preprocess the data
selected_features = ['Pclass', 'Sex', 'Age', 'SibSp', 'Parch', 'Fare']
data = data[selected_features + ['Survived']]
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1}) # Encode categorical variable
data = data.dropna() # Remove rows with missing values

X = data[selected_features] # Features
y = data['Survived']       # Target

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

 <ipython-input-5-1ca1b3f78378>:4: SettingWithCopyWarning:

A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy.
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1}) # Encode categorical variable

Step 3. Train the Decision Tree Model

Use a library like sklearn to create and train the model.

Experiment with parameters like criterion (e.g., Gini or Entropy) to find the best configuration.

```
# Step 3: Initialize and train the Decision Tree model
model = DecisionTreeClassifier(criterion='gini', max_depth=5, random_state=42)
model.fit(X_train, y_train)
```

 ▾ DecisionTreeClassifier ⓘ ?
DecisionTreeClassifier(max_depth=5, random_state=42)

Step 4. Evaluate the Model

Assess performance using metrics like accuracy, precision, recall, and F1-score. Visualize the Decision Tree for interpretability.

```
# Step 4: Make predictions
y_pred = model.predict(X_test)

# Display predictions for the first 10 test samples
```

```

print("\nSample predictions:")
print(f"Predicted: {y_pred[:10]}")
print(f"Actual:      {y_test.values[:10]}")

# Step 5: Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
print(f"\nAccuracy on test data: {accuracy * 100:.2f}%")

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

```



Sample predictions:

Predicted: [0 1 1 1 0 0 0 1 0 1]

Actual: [0 1 1 1 0 1 1 1 0 0]

Accuracy on test data: 72.03%

Classification Report:

	precision	recall	f1-score	support
0	0.78	0.75	0.76	87
1	0.63	0.68	0.66	56
accuracy			0.72	143
macro avg	0.71	0.71	0.71	143
weighted avg	0.72	0.72	0.72	143

Step 5. Visualize Predictions

Use confusion matrices, feature importances, or decision boundaries for insights.

```

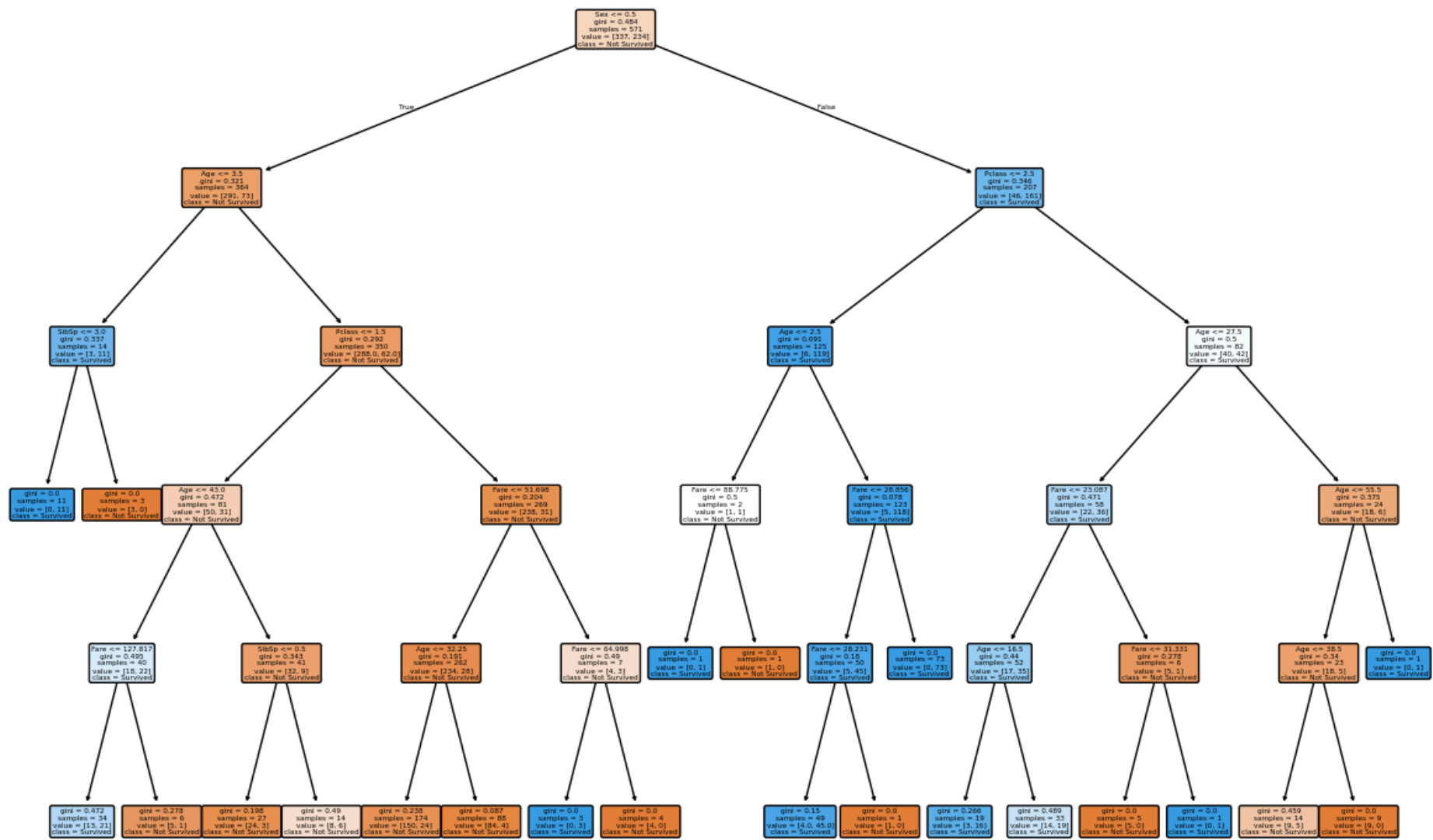
# Visualize the Decision Tree
plt.figure(figsize=(15, 10))
plot_tree(model, feature_names=selected_features, class_names=['Not Survived', 'Survived'], filled=True, rounded=True)
plt.title('Decision Tree Visualization')
plt.show()

# Confusion Matrix
conf_matrix = confusion_matrix(y_test, y_pred)
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues', xticklabels=['Not Survived', 'Survived'], yticklabels=['Not Survived', 'Survived'])
plt.title('Confusion Matrix')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()

```

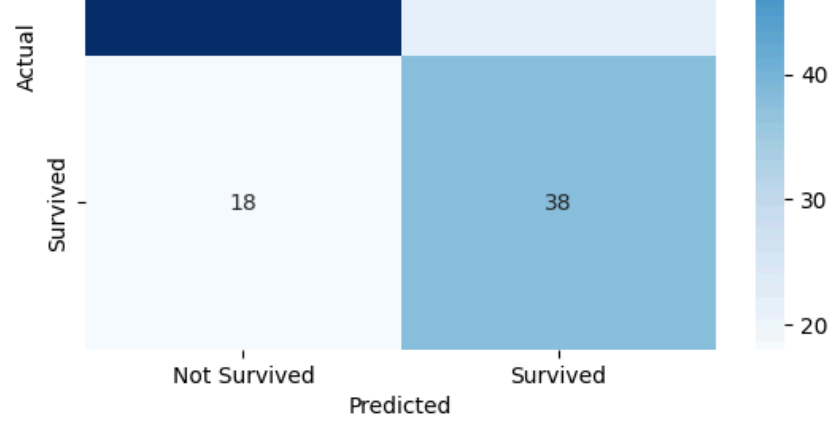



Decision Tree Visualization



Confusion Matrix





Exp 8: Implement KNN model to classify the target in given dataset.

The K-Nearest Neighbors (KNN) algorithm is a simple, versatile, and powerful method used in classification and regression tasks. It is a supervised learning algorithm that relies on the proximity of data points to classify or predict the target variable.

KNN works on the principle that similar data points tend to be in close proximity to each other in the feature space.

When predicting the class of a new data point, KNN identifies the K nearest neighbors (the K closest data points) from the training dataset and assigns the most frequent class (for classification) or the average (for regression) of these neighbors as the prediction.

For Classification (KNN classification):

Select a value for K: The first step is to choose the number of neighbors (K) that will be considered when making the prediction. A smaller K makes the model sensitive to noise, while a larger K makes the model more generalized.

Calculate the distance: KNN computes the distance between the new data point and all other points in the dataset. Common distance metrics include:

- **Euclidean Distance:**

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

- **Manhattan Distance:**

$$d = |x_1 - x_2| + |y_1 - y_2|$$

Find the K nearest neighbors: After computing the distance, the algorithm identifies the K nearest neighbors of the data point based on the smallest distances.

Assign the class: In classification, the class label of the new data point is determined by the majority class among the K neighbors. If there's a tie, some methods use a distance-weighted voting mechanism to break the tie.

For example, if K=5 and the 5 nearest neighbors have the classes: [0, 0, 1, 1, 1], the class of the new point will be 1 (majority vote).

✓ Step 1: Import necessary libraries

```
# Step 1: Import necessary libraries
import numpy as np
import pandas as pd
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns
```

✓ Step 2: Load the Wine dataset

Load the Wine Dataset:

The `load_wine()` function from `sklearn.datasets` loads the Wine dataset, which contains 178 samples with 13 features each (such as alcohol content, color intensity, flavonoids, etc.) and a target variable (`y`), which indicates the class of the wine (three classes: 0, 1, or 2).

```
# Step 2: Load the Wine dataset
wine = load_wine()
X = wine.data # Features (13 features about wines)
y = wine.target # Target (wine class: 0, 1, or 2)
```

Step 3: Convert the dataset into a DataFrame for easier manipulation (optional)

✓ Step 4: Understand the data

Convert to DataFrame (Optional):

We convert the dataset into a pandas DataFrame for easier inspection and manipulation.

```
# Step 3: Convert the dataset into a DataFrame for easier manipulation (optional)
data = pd.DataFrame(X, columns=wine.feature_names)
data['target'] = y
```

```
# Step 4: Understand the data
print(data.info()) # Check the structure of the dataset
print(data.head()) # Preview the first few rows
```

```
↗ <class 'pandas.core.frame.DataFrame'>
RangeIndex: 178 entries, 0 to 177
Data columns (total 14 columns):
#   Column                Non-Null Count  Dtype
---  -
0   alcohol                178 non-null    float64
1   malic_acid             178 non-null    float64
2   ash                    178 non-null    float64
```

3	alkalinity_of_ash	178 non-null	float64
4	magnesium	178 non-null	float64
5	total_phenols	178 non-null	float64
6	flavanoids	178 non-null	float64
7	nonflavanoid_phenols	178 non-null	float64
8	proanthocyanins	178 non-null	float64
9	color_intensity	178 non-null	float64
10	hue	178 non-null	float64
11	od280/od315_of_diluted_wines	178 non-null	float64
12	proline	178 non-null	float64
13	target	178 non-null	int64

dtypes: float64(13), int64(1)

memory usage: 19.6 KB

None

	alcohol	malic_acid	ash	alkalinity_of_ash	magnesium	total_phenols	\
0	14.23	1.71	2.43	15.6	127.0	2.80	
1	13.20	1.78	2.14	11.2	100.0	2.65	
2	13.16	2.36	2.67	18.6	101.0	2.80	
3	14.37	1.95	2.50	16.8	113.0	3.85	
4	13.24	2.59	2.87	21.0	118.0	2.80	

	flavanoids	nonflavanoid_phenols	proanthocyanins	color_intensity	hue	\
0	3.06	0.28	2.29	5.64	1.04	
1	2.76	0.26	1.28	4.38	1.05	
2	3.24	0.30	2.81	5.68	1.03	
3	3.49	0.24	2.18	7.80	0.86	
4	2.69	0.39	1.82	4.32	1.04	

	od280/od315_of_diluted_wines	proline	target
0	3.92	1065.0	0
1	3.40	1050.0	0
2	3.17	1185.0	0
3	3.45	1480.0	0
4	2.93	735.0	0

✓ Step 5: Split the dataset into training and testing sets (80% train, 20% test)

Splitting the Data:

The dataset is split into training and testing sets using `train_test_split()`. 80% of the data will be used for training, and 20% will be used for testing.

```
# Step 5: Split the dataset into training and testing sets (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

✓ Step 6: Feature scaling (important for distance-based models like KNN)

Feature Scaling:

We apply StandardScaler to scale the features, ensuring they have a mean of 0 and a standard deviation of 1. This is important for distance-based algorithms like KNN.

```
# Step 6: Feature scaling (important for distance-based models like KNN)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

✓ Step 7: Train the KNN model

Training the KNN Model:

We create a KNN classifier object with n_neighbors=5 (this means that the classifier will consider the 5 nearest neighbors to classify a wine sample). The model is trained with `knn.fit(X_train, y_train)`.

```
# Step 7: Train the KNN model
# Choosing K = 5 for this example
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
```



▼ KNeighborsClassifier ⓘ ?
KNeighborsClassifier()

✓ Step 8: Make predictions

Making Predictions:

The trained model makes predictions for the test set using `knn.predict(X_test)`.

```
# Step 8: Make predictions
y_pred = knn.predict(X_test)
```

✓ Step 9: Evaluate the model

Model Evaluation:

The accuracy of the model is computed using `accuracy_score`, and a confusion matrix is displayed using `confusion_matrix`, which shows the performance of the classifier in more detail.

```
# Step 9: Evaluate the model
# Accuracy score
print(f'Accuracy: {accuracy_score(y_test, y_pred)}')

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
print('Confusion Matrix:')
print(cm)
```



Accuracy: 0.9444444444444444

Confusion Matrix:

```
[[14  0  0]
 [ 1 12  1]
 [ 0  0  8]]
```

✓ Step 10: Visualize the Confusion Matrix

Confusion Matrix Visualization: You will see a heatmap, where:

The diagonal elements represent correct predictions for each class (e.g., 17 for class 0, 17 for class 1, and 15 for class 2).

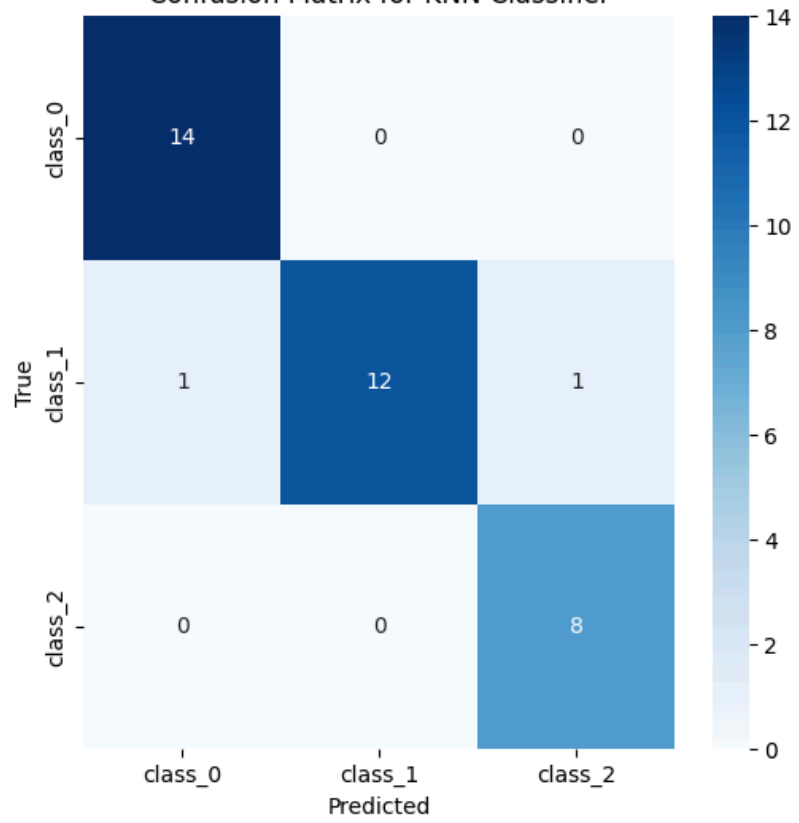
Off-diagonal elements represent misclassifications.

```
# Step 10: Visualize the Confusion Matrix
plt.figure(figsize=(6,6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=wine.target_names, yticklabels=wine.target_names)
plt.title('Confusion Matrix for KNN Classifier')
plt.xlabel('Predicted')
plt.ylabel('True')
plt.show()
```

```
# Step 11: Display some predictions
print("\nSample predictions:")
for i in range(10):
    print(f"True class: {y_test[i]}, Predicted class: {y_pred[i]}")
```



Confusion Matrix for KNN Classifier



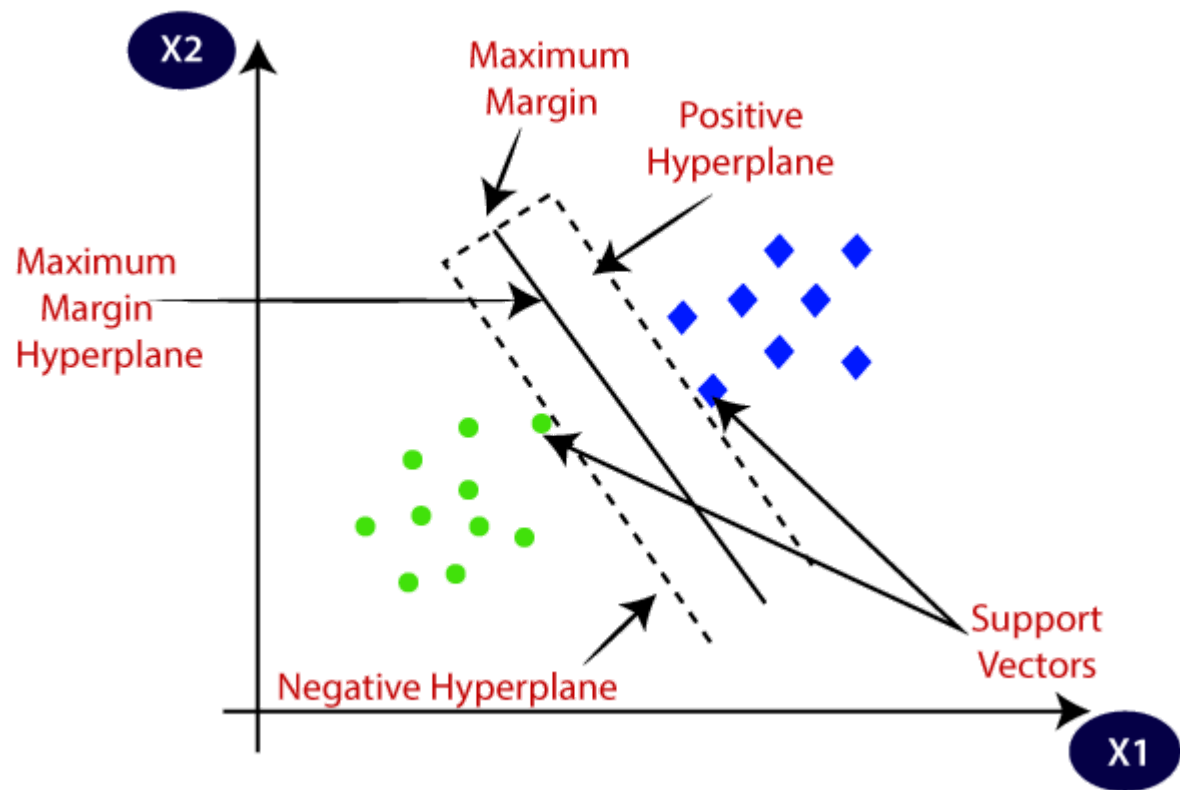
Sample predictions:

True class: 0, Predicted class: 0
True class: 0, Predicted class: 0
True class: 2, Predicted class: 2
True class: 0, Predicted class: 0
True class: 1, Predicted class: 1
True class: 0, Predicted class: 0
True class: 1, Predicted class: 1
True class: 2, Predicted class: 2
True class: 1, Predicted class: 1
True class: 2, Predicted class: 2

Exp 9: Build SVM model with various kernels and select best kernel for given dataset.

Support Vector Machine (SVM) is a powerful supervised learning algorithm used for classification, regression, and outlier detection tasks. It is particularly effective in high-dimensional spaces and when the number of features is greater than the number of samples. SVM is based on the concept of finding a hyperplane that best separates data points into different classes.

1. **Hyperplane** A hyperplane is a decision boundary that separates different classes in the feature space. For a binary classification problem, the SVM algorithm aims to find the optimal hyperplane that maximizes the margin between the two classes.
2. **Margin** The margin is the distance between the hyperplane and the nearest data points of each class, called support vectors. SVM tries to maximize this margin to ensure the classifier is as robust as possible.
3. **Support Vectors** Support vectors are the data points closest to the hyperplane. They are critical because the position of the hyperplane is determined by these points.



Step-by-Step Procedure:

Load the Dataset: We'll load the Wine dataset.

Preprocess the Data: We'll standardize the dataset.

Train SVM with Different Kernels: We'll use multiple kernels such as linear, polynomial, RBF, and sigmoid.

Evaluate the Models: We'll evaluate each kernel's performance using classification metrics.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn import datasets
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.preprocessing import StandardScaler
```

```
# Step 1: Load the Wine dataset
wine = datasets.load_wine()
X = wine.data
y = wine.target
```

Data Splitting: We split the dataset into training and testing sets with a 70/30 ratio using `train_test_split()`.

```
# Step 2: Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

Data Standardization: Standardize the dataset using `StandardScaler()` to ensure the features have zero mean and unit variance.

```
# Step 3: Standardize the dataset (important for SVM)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

SVM Model: The `SVC()` function is used to initialize the Support Vector Classifier. We apply different kernels (linear, poly, sigmoid) and train the model using `.fit()`.

Model Evaluation: For each kernel, the model is evaluated using `classification_report()` and `confusion_matrix()` to assess the performance.

```
# Step 4: Define a function to train SVM with different kernels
def train_svm(kernel):
    model = SVC(kernel=kernel)
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    # Step 5: Evaluate the model
    print(f"Kernel: {kernel}")
    print("Classification Report:")
    print(classification_report(y_test, y_pred))
    print("Confusion Matrix:")
```

```
print(confusion_matrix(y_test, y_pred))
print("-" * 50)
```

```
# Step 6: Train SVM with different kernels
kernels = ['linear', 'poly', 'sigmoid']
for kernel in kernels:
    train_svm(kernel)
```

```
⇒ Kernel: linear
Classification Report:
              precision    recall  f1-score   support

     0           1.00        1.00        1.00        19
     1           1.00        0.95        0.98        21
     2           0.93        1.00        0.97        14

 accuracy          0.98
 macro avg          0.98
weighted avg          0.98
```

Confusion Matrix:

```
[[19  0  0]
 [ 0 20  1]
 [ 0  0 14]]
```

Kernel: poly

```
Classification Report:
              precision    recall  f1-score   support

     0           1.00        0.89        0.94        19
     1           0.91        1.00        0.95        21
     2           1.00        1.00        1.00        14

 accuracy          0.96
 macro avg          0.97
weighted avg          0.97
```

Confusion Matrix:

```
[[17  2  0]
 [ 0 21  0]
 [ 0  0 14]]
```

Kernel: sigmoid

Classification Report:

	precision	recall	f1-score	support
0	0.95	1.00	0.97	19
1	1.00	0.95	0.98	21
2	1.00	1.00	1.00	14
accuracy			0.98	54
macro avg	0.98	0.98	0.98	54
weighted avg	0.98	0.98	0.98	54

Confusion Matrix:

```
[[19  0  0]
 [ 1 20  0]
 [ 0  0 14]]
```

Exp 10 : Build Random Forest model and apply on given dataset. Evaluate the model with suitable metrics.

Random Forest is an ensemble learning method used for classification, regression, and other tasks. It operates by constructing multiple decision trees during training and combines their outputs to make predictions. It's a powerful and widely-used machine learning algorithm due to its simplicity, flexibility, and ability to handle complex datasets.

A decision tree splits data into branches based on feature values to create a tree structure. Each leaf represents a predicted outcome (class label or value). While decision trees are easy to interpret, they tend to overfit.

Ensemble Learning

Random Forest is a type of ensemble method that aggregates predictions from multiple models (decision trees) to improve accuracy and generalizability.

Build multiple decision trees during training. Use random subsets of data and features to train each tree. Combine predictions using:

Majority voting for classification. Averaging predictions for regression.

Bootstrap Sampling (Bagging):

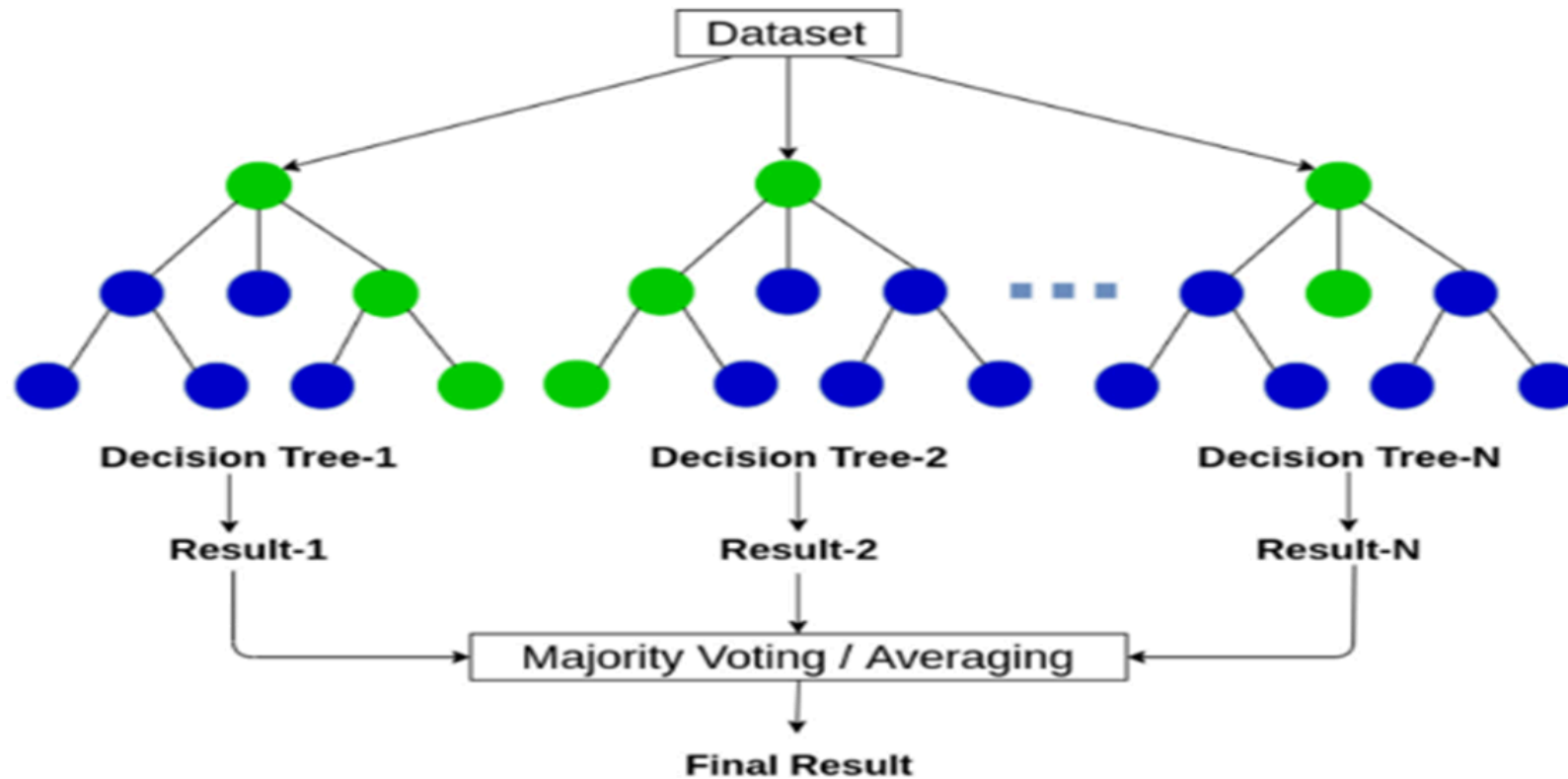
From the original dataset, create multiple subsets by random sampling with replacement. Each tree is trained on a different subset of data.

Feature Randomness:

At each split in a tree, a random subset of features is selected as candidates for splitting. This adds diversity to the trees and reduces correlation between them. Combining Predictions:

For classification: The forest aggregates the predictions of individual trees by majority voting. For regression: The forest averages the predictions of individual trees.

Random Forest



Step 1: Import Libraries

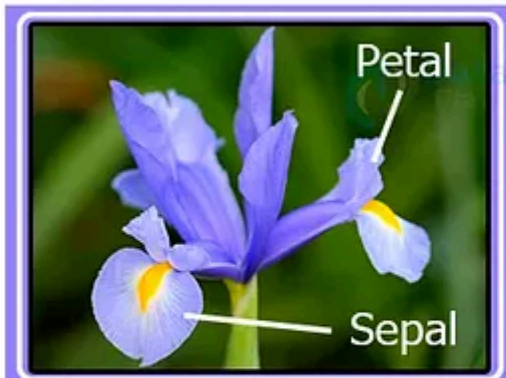
Before building the model, import the necessary libraries.

```
# Step 1: Import libraries
import pandas as pd
import numpy as np
```

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
import matplotlib.pyplot as plt
import seaborn as sns
```

Step 2: Load the Dataset

Load your dataset into a pandas DataFrame.



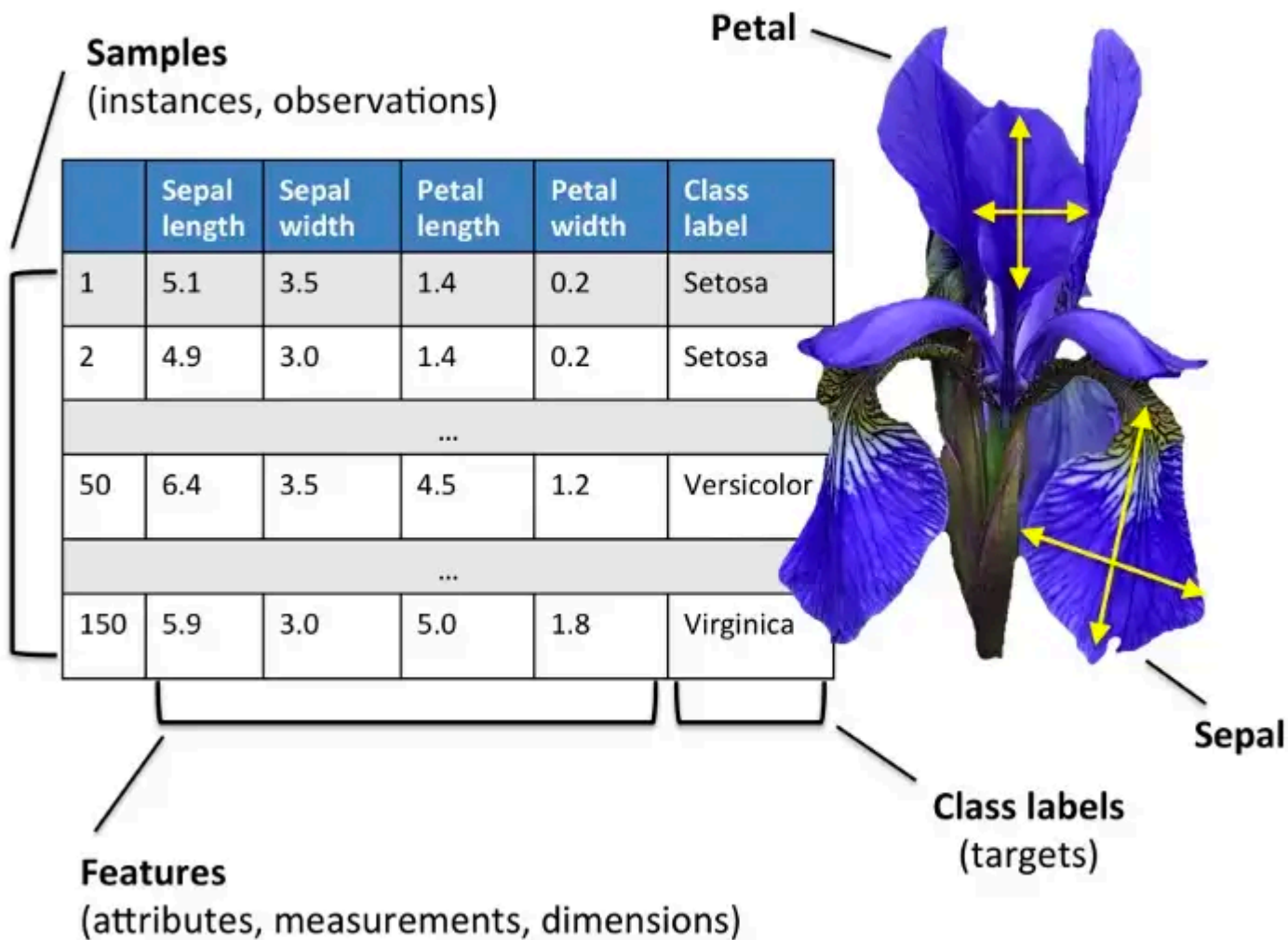
Iris Versicolor



Iris Setosa



Iris Virginica



```
# Step 2: Load the dataset
iris = load_iris() # Load the Iris dataset
data = pd.DataFrame(data=iris.data, columns=iris.feature_names) # Create DataFrame
```

```
data['target'] = iris.target # Add target column
```

```
# Display first few rows
print("Dataset Sample:")
print(data.head())
```

```
Dataset Sample:
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  \
0                5.1                3.5                1.4                0.2
1                4.9                3.0                1.4                0.2
2                4.7                3.2                1.3                0.2
3                4.6                3.1                1.5                0.2
4                5.0                3.6                1.4                0.2

   target
0       0
1       0
2       0
3       0
4       0
```

Step 3: Preprocess the Data

Handle Missing Values: If your dataset has missing values, handle them appropriately by imputing or dropping.

Encode Categorical Variables: Use one-hot encoding or label encoding for categorical features.

Separate Features and Target Variable: Identify the target variable and split the dataset into features (X) and target (y).

```
# Step 3: Preprocess the data
# No missing values or categorical data in Iris dataset, so no preprocessing is needed here.

# Separate features and target
X = data.drop('target', axis=1) # Features
y = data['target'] # Target
```

Step 4: Split the Dataset

Divide the data into training and testing sets.

```
# Step 4: Split the dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Step 5: Train the Random Forest Model

Fit a Random Forest classifier to the training data.

```
# Step 5: Train the Random Forest model
rf_model = RandomForestClassifier(n_estimators=100, random_state=42) # Initialize model
rf_model.fit(X_train, y_train) # Train the model
```



```
RandomForestClassifier
RandomForestClassifier(random_state=42)
```

Step 6: Make Predictions

Use the trained model to make predictions on the test set.

```
# Step 6: Make predictions
y_pred = rf_model.predict(X_test)
```

Step 7: Evaluate the Model

Accuracy Score:

Measure the proportion of correctly classified samples. Confusion Matrix:

Display the confusion matrix for true vs. predicted labels. Classification Report:

Provide precision, recall, F1-score, and support for each class.

```
# Step 7: Evaluate the model
# Accuracy score
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.2f}")
```

```
# Confusion matrix
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=iris.target_names, yticklabels=iris.target_names)
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.show()

# Classification report
print("Classification Report:\n", classification_report(y_test, y_pred, target_names=iris.target_names))
```



Accuracy: 1.00

Confusion Matrix

